

Automatisierte Microsegmentierungsstrategie basierend auf Digital Twin Netzwerk-Mapping

IP6 Bachelor Thesis

Windisch, August 2026



Studenten

Martin Vogt

Auftraggeber

Christopher Scherb, Luc Heitz

Fachhochschule Nordwestschweiz, Hochschule für Informatik

Abstract

Die Komplexität und Dynamik moderner IT-Infrastrukturen nehmen kontinuierlich zu, wodurch die manuelle Planung und Umsetzung einer konsistenten Netzwerksegmentierung zunehmend erschwert wird. Diese Arbeit untersucht, ob ein graphbasierter Digital-Twin-Ansatz mit abstrakten Kommunikationsanforderungen in eine VLAN-basierte Mikrosegmentierung überführt werden kann.

Zu diesem Zweck wird ein Prototyp entwickelt, der Hosts, Netzwerkgeräte und deren Verbindungen als Graph modelliert. Erlaubte und verbotene Kommunikationsanforderungen werden in Form von High-Level-Constraints als ALLOW- und DENY-Regeln abgebildet. Die Berücksichtigung der zugrunde liegenden Topologie stellt bei der Berechnung der Netzwerksegmentierung ein zentrales Element dar. Zur Lösung des Segmentierungsproblems werden zwei voneinander unabhängige Verfahren implementiert. Ein schrittweise arbeitender heuristischer Ansatz wird einem formalen Ansatz auf Basis des SMT-Solvers Z3 gegenübergestellt.

Die Evaluation erfolgt anhand von 72 synthetisch erzeugten Netzwerkszenarien mit Star-, Tree- und Mesh-Topologien, unterschiedlichen Netzwerkgrößen sowie verschiedenen Constraint-Dichten. Die Resultate zeigen einen klaren Zielkonflikt zwischen Geschwindigkeit und Zuverlässigkeit. Die Heuristik benötigt bei erfolgreichen Durchläufen nur wenig Rechenzeit, erzeugt jedoch lediglich für die Hälfte der untersuchten Szenarien eine gültige Segmentierung. Der Z3-basierte Ansatz findet hingegen für sämtliche untersuchten erfüllbaren Szenarien eine gültige Lösung, verwendet dabei jedoch häufig mehr VLANs.

Die Arbeit zeigt, dass ein Digital-Twin-Modell in Kombination mit einer graphbasierten Pfadanalyse und formal definierten Constraints eine geeignete Grundlage für die automatisierte Mikrosegmentierung bildet.

Keywords:

Mikrosegmentierung, Digital Twin, Netzwerksegmentierung, Graphmodellierung, Constraint Solving, SMT, Z3, VLAN

Dank

Mein besonderer Dank gilt Christopher Scherb und Luc Heitz für die fachliche Betreuung dieser Bachelorarbeit. Ihre konstruktiven Rückmeldungen und die regelmässigen Gespräche haben wesentlich zur Entwicklung und Ausarbeitung dieser Arbeit beigetragen.

Ebenfalls danke ich meinem beruflichen und persönlichen Umfeld für die Unterstützung und das Verständnis während der intensiven Bearbeitungszeit. Insbesondere danke ich meiner Partnerin Stefanie für ihre Geduld, ihre Motivation und den Rückhalt, den sie mir während des Projekts gegeben hat.

Inhaltsverzeichnis

Abbildungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung der Arbeit	2
1.4 Forschungsfrage	2
1.5 Aufbau der Arbeit	3
1.6 Related work	3
1.6.1 Datengetriebene Segmentierung	3
1.6.2 Formale Netzwerksegmentierung	3
1.6.3 Constraint-basierte Security-Design-Synthese	4
2 Grundlagen	5
2.1 Netzwerksegmentierung & Microsegmentierung	5
2.2 Graphbasierte Modellierung	6
2.3 Digital Twin Konzepte	6
2.4 Constraint Solving (Z3)	7
3 Stand der Technik	8
3.1 Bestehende Segmentierungsansätze	8
3.2 Graphbasierte Ansätze	9
3.3 Vergleich der Ansätze	10
4 Systemdesign	11
4.1 Gesamtarchitektur	11
4.2 Datenmodelle	12
4.2.1 Constraint Modell	12
4.2.2 Netzwerkmodell (Digital Twin)	14
4.3 Graph Engine	16
4.4 Segmentierungsansätze	17
4.4.1 Heuristischer Ansatz	17
4.4.2 Solver-basierter Ansatz (Z3)	17
4.5 Gruppenmodellierung	18
4.6 Systemkomponenten (Web-Editor)	19

5	Implementierung	20
5.1	Technologiewahl	20
5.2	Projektstruktur	21
5.3	Umsetzung der Kernkomponenten	21
5.3.1	Graph Builder	21
5.3.2	Graph Engine	22
5.3.3	Heuristic Solver	23
5.3.4	Z3 Solver	24
5.3.5	Datenimport / Export	26
6	Evaluation	29
6.1	Ziel der Evaluation	29
6.2	Versuchsaufbau	29
6.3	Metriken	32
6.4	Ergebnisse	32
6.5	Analyse	35
6.6	Diskussion der Ergebnisse	35
7	Diskussion	37
7.1	Stärken des Ansatzes	37
7.2	Limitationen	38
7.3	Herausforderungen	39
8	Fazit und Ausblick	40
8.1	Zusammenfassung	40
8.2	Beantwortung der Forschungsfrage	40
8.3	Ausblick	41
	Quellenverzeichnis	42
9	Glossar	43
	Eigenständigkeitserklärung	45

A	Anhang	47
A.1	Projektvereinbarung	48
A.2	Meetingprotokoll vom 25. März 2026	55
A.3	Meetingprotokoll vom 7. April 2026	57
A.4	Meetingprotokoll vom 21. April 2026	59
A.5	Meetingprotokoll vom 12. Mai 2026	62
A.6	Meetingprotokoll vom 7. Juli 2026	65
A.7	Meetingprotokoll vom 4. August 2026	67

Abbildungsverzeichnis

4.1	Visualisierung der Verarbeitungspipeline	11
4.2	Einfaches Netzwerkszenario	15
5.1	Einfaches Beispiel der Graphenvisualisierung	28
6.1	Star-Topologie mit 120 Hosts	30
6.2	Tree-Topologie mit 120 Hosts	31
6.3	Mesh-Topologie mit 120 Hosts	31
6.4	Laufzeitvergleich der Segmentierungsansätze	33
6.5	Einfluss der Anzahl Constraints auf die Laufzeit	33
6.6	Vergleich der verwendeten VLANs	34
6.7	Vergleich der Robustheit der Segmentierungsansätze	34

Listings

1	Beispiel eines einfachen Netzwerkszenarios	14
2	Definition der Knoten und Knotentypen	21
3	Definition der Kanten	22
4	Implementierung der Graphstruktur	22
5	Initialisierung der Graph Engine	22
6	Bestimmung des kürzesten Pfades zwischen zwei Knoten	23
7	Datenstruktur zur Speicherung der Segmentierungsergebnisse	23
8	Verarbeitung der ALLOW-Constraints im HeuristicSynthesizer	24
9	Definition der booleschen Entscheidungsvariablen	24
10	Erzeugung der Bedingungen für die möglichen VLANs	25
11	Verknüpfung der VLAN-Bedingungen durch eine Disjunktion	25
12	Strukturierte Ausgabe des Verarbeitungsprozesses und der Segmentierungsergebnisse	27
13	Beispielhafte Anzahl der Constraints in den Experimenten	29
14	Verkürzter Auszug der erfassten Experimentergebnisse	32

1 Einleitung

1.1 Motivation

Die IT-Umgebungen moderner Unternehmen werden zunehmend komplexer. Netzwerke bestehen längst nicht mehr nur aus statischen Systemen, sondern auch dynamische Komponenten wie virtuelle Maschinen, Container, Cloud-Dienste sowie IoT-Geräte sind mittlerweile Standard. Diese heterogenen Systeme sind über eine Vielzahl von Kommunikationsbeziehungen miteinander verbunden. Die Strukturen werden zunehmend schwerer nachvollziehbar und klassische Ansätze zur Netzwerksegmentierung stossen dabei an ihre Grenzen. Die zunehmende Dynamik moderner Netzwerke erschwert eine statische Sicherheitskonfiguration zusätzlich, da Ressourcen laufend hinzugefügt, entfernt oder verändert werden können. Für solche Umgebungen werden adaptive und automatisierte Segmentierungsverfahren benötigt Alabbad, 2021.

Eines der wichtigsten Sicherheitsinstrumente zur Verringerung der Angriffsfläche in modernen Computernetzwerken ist die Microsegmentierung. Ziel ist es, Kommunikationsbeziehungen zwischen Systemen so restriktiv wie möglich zu gestalten und nur tatsächlich notwendige Verbindungen zu erlauben. In der Praxis werden entsprechende Regeln meist manuell zusammengetragen und auf Netzwerkkomponenten wie Firewalls oder Switches in Form von VLANs aufgespielt. Dieser Prozess ist nicht nur zeitaufwendig, sondern auch fehleranfällig und schwer skalierbar. Vor allem in grösseren und dynamischen Netzwerken führt dies zu inkonsistenten Regelwerken, die entweder zu restriktiv sind und Funktionalität einschränken oder zu permissiv und somit sicherheitskritisch bleiben Alabbad, 2021; Smeriga und Jirsik, 2019.

Vor diesem Hintergrund ergibt es Sinn, automatisierte Verfahren zur Unterstützung der Netzwerksegmentierung zu prüfen. Speziell Ansätze, die Netzwerkstrukturen modellieren, Kommunikationsbeziehungen analysieren und daraus konsistente Segmentierungsvorlagen ableiten, bieten ein hohes Potenzial zur Bewältigung der genannten Herausforderungen.

1.2 Problemstellung

Die praktische Umsetzung von Microsegmentierung in modernen Netzwerken ist komplexer, als der erste Blick auf das Problem vermuten lässt. Das eigentliche Ziel kann mit - nur notwendige Kommunikation zulassen und alles andere unterbinden - einfach beschrieben werden; die konkrete Anwendung dieses Prinzips wirft jedoch eine Reihe technischer Herausforderungen auf. Ein zentrales Problem besteht in der Modellierung des Netzwerks selbst. Infrastrukturen in der realen Welt sind nicht statisch, sondern verändern sich kontinuierlich. Bestehende Systeme werden entfernt und neue Komponenten kommen hinzu. Oft existieren zwischen Anwendungen vielfältige Abhängigkeiten, die häufig nicht vollständig dokumentiert sind. Eine konsistente und ausreichend detaillierte Repräsentation dieser Strukturen zu erstellen, ist daher alles andere als trivial.

Teilweise ist unklar, welche Kommunikationsbeziehungen tatsächlich erforderlich sind. Es ist weit verbreitet, dass sich im Laufe der Zeit Verbindungen etablieren, deren ursprünglicher Zweck nicht mehr nachvollziehbar ist. Der Grund dafür ist oft, dass bei Rückbauten von Infrastrukturen und Anwendungen die Konnektivität eine zweitrangige Rolle spielt. Genau an dieser Stelle entsteht ein Spannungsfeld zwischen Funktionalität und Sicherheit. Besonders herausfordernd wird die Situation, wenn sich Anforderungen widersprechen. So kann es vorkommen, dass zwei Systeme miteinander kommunizieren müssen, gleichzeitig aber voneinander isoliert sein sollen. Konflikte dieser Art lassen sich manuell nur schwer erkennen oder in konsistenter Form auflösen. In bestimmten Fällen existiert sogar keine gültige Lösung, was eine explizite Konflikterkennung erforderlich macht. Formale Syntheseverfahren bieten in diesem Zusammenhang den Vorteil,

widersprüchliche Anforderungen explizit als nicht erfüllbar zu erkennen und die beteiligten Bedingungen für eine weitere Analyse einzugrenzen Rahman und Al-Shaer, 2013.

Ein weiterer Aspekt ist die Skalierbarkeit. Bei einer paarweisen Betrachtung der Kommunikationsbeziehungen kann die Anzahl möglicher Flows quadratisch mit der Anzahl der Hosts wachsen, wodurch zugleich die Zahl der zu verarbeitenden Bedingungen zunimmt Rahman und Al-Shaer, 2013. Algorithmen, die zur Segmentierung eingesetzt werden, müssen daher in der Lage sein, auch grössere Netzwerke effizient zu verarbeiten, ohne dass dabei Genauigkeit oder Konsistenz verloren gehen.

Daraus ergibt sich eine zentrale Problemstellung für diese Arbeit: Es soll ein Verfahren entwickelt werden, das Netzwerke strukturiert modelliert, Kommunikationsbeziehungen analysiert und daraus konsistente sowie möglichst sichere Segmentierungsstrategien ableitet. Skalierbarkeit, Anwendbarkeit, Korrektheit und auch die Konflikterkennung sollen berücksichtigt werden.

1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Entwicklung eines Prototypen zur automatischen Mikrosegmentierung von Netzwerken. Dabei soll untersucht werden, wie sich Kommunikationsbeziehungen analysieren und in konsistente, sicherheitsorientierte Segmentierungsstrategien überführen lassen. Im Zentrum steht die Modellierung des Netzwerks als Digital Twin. Diese Repräsentation ermöglicht es, Systeme und deren Verbindungen strukturiert abzubilden und für weitere Analysen nutzbar zu machen. Daraus werden Verfahren zur automatischen Synthese von Segmentierungsregeln entwickelt, die nur absolut notwendige Kommunikation erlauben und eine möglichst hohe Isolation zwischen den Systemen gewährleisten. Hierbei werden ein heuristischer Ansatz sowie ein Constraint-basierter Ansatz kombiniert, um sowohl Effizienz als auch Korrektheit sicherzustellen.

Die Lösung wird prototypisch implementiert und anhand synthetischer Netzwerke evaluiert, um ihre Funktionsweise sowie ihre Grenzen zu untersuchen.

1.4 Forschungsfrage

Im Zentrum dieser Arbeit steht die folgende Forschungsfrage:

Wie kann die Mikrosegmentierung von Netzwerken durch eine Kombination aus Digital Twin Modellierung und automatisierter Regel-Synthese effektiv und konsistent umgesetzt werden?

Zur weiteren Konkretisierung werden folgende Teilfragen betrachtet:

- Wie können Netzwerke und deren Kommunikationsbeziehungen geeignet modelliert werden, um eine automatisierte Analyse zu ermöglichen?
- Inwiefern unterscheiden sich heuristische und Constraint basierte Ansätze hinsichtlich Korrektheit, Skalierbarkeit und Praktikabilität?
- Wie lassen sich aus den gewonnenen Modellen konsistente und sicherheitsorientierte Segmentierungsregeln ableiten?

1.5 Aufbau der Arbeit

Nach der Einleitung und der Einordnung verwandter Arbeiten werden in Kapitel 3 die notwendigen Grundlagen behandelt. Kapitel 4 beschreibt den Stand der Technik, bevor Kapitel 5 das Systemdesign und Kapitel 6 die konkrete Implementierung vorstellen. Anschliessend werden in Kapitel 7 die Experimente und deren Ergebnisse ausgewertet. Kapitel 8 diskutiert Stärken, Limitationen und Herausforderungen, während Kapitel 9 die Arbeit zusammenfasst, die Forschungsfrage beantwortet und einen Ausblick gibt.

1.6 Related work

Dieses Kapitel ordnet die vorliegende Arbeit in bestehende Forschungsansätze zur automatisierten Netzwerksegmentierung ein. Betrachtet werden datengestützte Verfahren, formale Segmentierungsmodelle sowie Constraints-basierte Ansätze zur Synthese von Sicherheitskonfigurationen.

1.6.1 Datengetriebene Segmentierung

Smeriga und Jirsik untersuchen eine verhaltensbasierte Netzwerksegmentierung anhand von IP-Flow-Daten. Aus Merkmalen wie der Anzahl der Flows, Kommunikationspartner, Ports, Protokollen und Datenvolumen werden Verhaltensprofile für Hosts erstellt. Anschliessend werden Clustering-Verfahren wie K-Means und DBSCAN zur Erkennung potenzieller Segmente eingesetzt Smeriga und Jirsik, 2019. Im Gegensatz dazu leitet die vorliegende Arbeit die Segmentierung nicht aus beobachteten Verkehrsverhalten ab. Die Kommunikationsanforderungen werden explizit als High-Level-Constraints vorgegeben und mit einer bekannten Netzwerktopologie verknüpft.

1.6.2 Formale Netzwerksegmentierung

Mhaskar, Alabbad und Khedri formulieren Netzwerksegmentierung auf Basis von Access-Control-Policies formal. Ressourcen mit ähnlichen Sicherheitsanforderungen werden zu Segmenten zusammengefasst und durch strategisch platzierte Firewalls geschützt. Aus den Policies werden sowohl eine robuste Netzwerktopologie als auch die benötigten Firewall-Regeln abgeleitet Mhaskar et al., 2021.

Alabbad erweitert diesen Ansatz insbesondere hinsichtlich dynamischer Netzwerke und untersucht eine adaptive Rekonfiguration von Software-Defined Networks bei Änderungen der Topologie, der Ressourcen oder der Sicherheitsrichtlinien Alabbad, 2021.

Während diese Ansätze eine Netzwerktopologie aus den Sicherheitsrichtlinien konstruieren, geht die vorliegende Arbeit von einer bestehenden Topologie aus und berechnet geeignete VLAN-Zuweisungen innerhalb dieses Digital-Twin-Modells.

1.6.3 Constraint-basierte Security-Design-Synthese

Rahman und Al-Shaer stellen mit ConfigSynth ein automatisiertes Framework zur Synthese von Netzwerksicherheitskonfigurationen vor. Als Eingaben dienen eine bestehende Netzwerktopologie, Isolationsanforderungen sowie Bedingungen bezüglich Nutzbarkeit und Bereitstellungskosten. Das resultierende Syntheseproblem wird in SMT überführt und mit Z3 gelöst Rahman und Al-Shaer, 2013.

Der in dieser Arbeit entwickelte Ansatz verwendet ebenfalls eine bestehende Topologie und einen SMT-Solver, beschränkt sich jedoch bewusst auf die VLAN-basierte Segmentierung anhand von ALLOW- und DENY-Constraints. Zusätzlich wird ein unabhängiges heuristisches Verfahren implementiert und mit der Solver-Lösung verglichen.

2 Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen erklärt, die für das Verständnis der weiteren Arbeit erforderlich sind. Zentrale Konzepte aus den Bereichen Netzwerksegmentierung, graphenbasierte Modellierung und formale Methoden zur Problemlösung werden weiter beschrieben.

Beginnend mit den Grundlagen der Netzwerksegmentierung und insbesondere der Microsegmentierung wird der erste Teil abgedeckt. Anschliessend werden Graphen als Modellierungswerkzeug für Netzwerke betrachtet. Darauf aufbauend werden Constraint-basierte Ansätze vorgestellt, die eine formale Beschreibung und Lösung von Segmentierungsproblemen ermöglichen. Ziel ist es, in diesem Kapitel die notwendigen Begriffe und Konzepte zu definieren, welche in die Systementwürfe der weiteren Kapitel einfließen.

2.1 Netzwerksegmentierung & Microsegmentierung

Das Segmentieren von Netzwerken ist ein Basiskonzept zur Erhöhung der Sicherheit und zur Strukturierung komplexer Infrastrukturen. Netzwerke in kleine, logisch getrennte Bereiche zu unterteilen, um Kommunikationsflüsse gezielt zu steuern und potenzielle Angriffe zu erschweren, ist dabei das Hauptziel. Ein klassischer Ansatz ist dabei, die Netzwerksegmentierung über die Aufteilung in Subnetze oder VLANs zu steuern. Dabei werden Systeme mit ähnlichen Anforderungen oder derselben organisatorischen Einheit in Gruppen eingeteilt, welche später als einzelne Segmente dienen. Die Kommunikation innerhalb solcher Segmente wird oft nicht oder nur reduziert kontrolliert. Netzwerksegmentierung folgt dabei dem Grundgedanken, Ressourcen mit vergleichbaren Sicherheitsanforderungen in gemeinsamen Zonen zusammenzufassen und Ressourcen mit unterschiedlichen Anforderungen voneinander zu trennen Mhaskar et al., 2021.

Das zentrale Mittel in diesem Kontext ist der Einsatz von Virtual Local Area Networks (VLANs). VLANs ermöglichen es, logische Netzwerke unabhängig von der physischen Infrastruktur zu definieren und bieten eine praktische Möglichkeit zur Segmentierung. Hinzu kommen Firewalls, um den Datenverkehr zwischen Segmenten auf der Basis definierter Regeln zu filtern. Diese Regeln legen fest, welche Kommunikationsbeziehungen erlaubt oder blockiert sind.

Diese Ansätze reichen in vielen Umgebungen aus, zeigen aber insbesondere bei dynamischen Netzwerken einige markante Einschränkungen. Die Segmentierung erfolgt oft grobgranular, wodurch innerhalb eines Segments meist ungefiltert zwischen den einzelnen Systemen kommuniziert werden kann. Dies kann im Falle eines Angriffs dazu führen, dass sich der Angreifer innerhalb eines Segments ungehindert ausbreiten kann.

Modernere Ansätze der Netzwerksegmentierung setzen daher auf eine weitaus feingranuläre Lösung. Dabei werden einzelne Kommunikationsbeziehungen zwischen den Komponenten auf System- oder Dienstebene gesteuert. Microsegmente können nur wenige Hosts mit vergleichbaren Berechtigungen oder sogar einzelne Anwendungen umfassen Smeriga und Jirsik, 2019. Damit kann das Prinzip der minimalen Berechtigung konsequent umgesetzt werden, sodass nur die tatsächlich erforderlichen Verbindungen zugelassen werden. Der Preis dafür ist meist ein Agent auf den Endgeräten und höhere Betriebsaufwände.

2.2 Graphbasierte Modellierung

Graphen sind ein etabliertes Modellierungswerkzeug, um Beziehungen in komplexen Systemen strukturiert abzubilden. Ein Graph besteht aus einer Menge von Knoten (Nodes) und Kanten (Edges), wobei Knoten in einem Netzwerk einzelne Endgeräte repräsentieren und Kanten die Beziehung zwischen diesen beschreiben. Auch formale Ansätze zur Security-Design-Synthese modellieren Netzwerke als Graphen, in denen Hosts und Netzwerkgeräte als Knoten sowie deren Verbindungen als Kanten dargestellt werden Mhaskar et al., 2021; Rahman und Al-Shaer, 2013. Ein grosser Vorteil von graphbasierten Modellen liegt in der Flexibilität. Neben der einfachen Erfassung von Komponenten als Nodes oder der physischen Verbindung als Kanten können weitere Informationen wie Protokolle und Ports ebenfalls an die Kanten angehängt werden.

Darüber hinaus ermöglichen Graphen den Einsatz zahlreicher Analyseverfahren. Unter anderem die Untersuchung von Abhängigkeiten, die Identifikation zentraler Knoten oder die Erkennung von Clustern innerhalb eines Netzwerks. Solche Methoden sind insbesondere für die Analyse von Kommunikationsmustern und die Ableitung von Segmentierungsstrategien interessant. Diverse Werkzeuge zur Modellierung von Graphen bringen diese Algorithmen bereits in effizienter Form als Toolkit mit.

In dieser Arbeit wird das Netzwerk als gerichteter Graph modelliert, in dem Kommunikationsbeziehungen zwischen Systemen explizit dargestellt werden. Die Umsetzung erfolgt unter Verwendung der Python-Bibliothek NetworkX, die eine flexible Erstellung, Verarbeitung und Analyse von Graphstrukturen ermöglicht. Diese dient als Grundlage für die weitere Verarbeitung und Analyse im Rahmen des entwickelten Prototyps.

2.3 Digital Twin Konzepte

Der Begriff „Digital Twin“ bezeichnet die digitale Abbildung eines realen Systems in einem virtuellen Modell. Ziel eines solchen Modells ist es, Struktur, Verhalten und den Zustand eines realen Systems möglichst vollständig abzubilden. Eingesetzt werden Digital Twins in weiten Bereichen der Industrie. Ein wesentlicher Vorteil solcher Modelle besteht darin, dass sie in der Regel bereits vor ihrem physischen Gegenstück als kostengünstige Simulation entwickelt werden. Dadurch dienen sie als Proof of Concept und ermöglichen erste Testläufe bereits in einer frühen Entwicklungsphase.

Im Kontext von Computernetzwerken bietet sich ein Digital Twin zur Entwicklung von Segmentierungsvorschlägen dahingehend an, dass neben der reinen Topologie auch zusätzliche Informationen wie Kommunikationsbeziehungen, Protokolle oder Abhängigkeiten zwischen Anwendungen simuliert und getestet werden können. Ein wesentlicher Vorteil eines solchen Modells liegt darin, dass Analysen nicht direkt auf dem produktiven System durchgeführt werden müssen. Stattdessen können Veränderungen, Optimierungen oder sicherheitsrelevante Massnahmen zunächst im digitalen Modell untersucht werden. Dadurch können Auswirkungen in der Simulation geprüft und potenzielle Risiken identifiziert werden. Insbesondere ermöglicht das Modell eine algorithmische Verarbeitung, die für die Ableitung von Segmentierungsstrategien erforderlich ist.

2.4 Constraint Solving (Z3)

Constraint Solving bezeichnet einen Ansatz zur Lösung von Problemen, bei dem eine Menge von Variablen unter gegebenen Einschränkungen (Constraints) so belegt wird, dass alle Bedingungen gleichzeitig erfüllt sind. Anstatt eine Lösung schrittweise zu konstruieren, wie dies bei dem heuristischen Ansatz getan wird, kann das Problem formal beschrieben und anschliessend von einem Solver automatisch gelöst werden.

Die zentrale Aufgabe für den Einsatz eines Solvers ist, das Problem in eine mathematisch oder logisch definierte Form zu überführen. Variablen repräsentieren dabei die zu bestimmenden Werte, während Constraints die Beziehungen und Einschränkungen zwischen Variablen festlegen. Eine gültige Lösung, also erfolgreiche Segmentierung, liegt genau dann vor, wenn alle Constraints erfüllt sind.

Aufbauend auf SAT wurde das Konzept des Satisfiability Modulo Theories (SMT) entwickelt. SMT erweitert das klassische SAT-Problem um zusätzliche Theorien, beispielsweise über ganze Zahlen, Mengen oder Funktionen. Dadurch können deutlich komplexere Probleme modelliert werden, die über reine boolesche Logik hinausgehen. SMT-Solver kombinieren dabei die Effizienz von SAT-Verfahren mit der Ausdrucksstärke formaler Theorien. Ein vergleichbarer Einsatz von SMT findet sich bei ConfigSynth, wo Netzwerktopologie, Isolationsanforderungen, Konnektivitätsanforderungen und weitere Geschäftsbedingungen gemeinsam als Constraint-System modelliert und mit Z3 gelöst werden Rahman und Al-Shaer, 2013.

Zur Umsetzung wird der SMT-Solver Z3 verwendet, der von Microsoft Research entwickelt wurde. Z3 ermöglicht die Modellierung und Lösung komplexer Constraint-Systeme und bietet Unterstützung für verschiedene Theorien, die für die Beschreibung von Netzwerksegmentierungsproblemen relevant sind.

3 Stand der Technik

Dieses Kapitel zeigt bestehende Ansätze zur Netzwerksegmentierung sowie deren Grenzen auf. Ziel ist es, einen Überblick über die gängigen Methoden zu geben und sie anhand ihrer Eignung für moderne Netzwerke zu bewerten.

Startend mit datengetriebenen Verfahren zur Segmentierung betrachten wir die Analyse von Netzwerkverkehr oder bestehenden Regeln. Anschliessend werden graphenbasierte Ansätze untersucht, die Netzwerke als strukturierte Modelle abbilden. Weiter erfolgt ein Vergleich der beiden Herangehensweisen, um deren Stärken und Schwächen herauszuarbeiten.

3.1 Bestehende Segmentierungsansätze

Ein verbreiteter Ansatz ist das sogenannte Rule Mining auf Basis von Netzwerkverkehr. Dabei werden Kommunikationsdaten analysiert und wiederholt auftretende Verbindungen markiert. So werden Kommunikationsmuster erkannt und Abhängigkeiten zwischen Systemen identifiziert. Daraus ergibt sich beispielsweise eine Sammlung von Interaktionen in der Form von „Host A kommuniziert mit Host B via Port 443“. IP-Flow-Monitoring ermöglicht eine passive Erfassung von Kommunikationspartnern, Ports, Protokollen, Verbindungsdauer und Datenvolumen, ohne dass ein direkter Zugriff auf die beobachteten Hosts erforderlich ist Smeriga und Jirsik, 2019. Ein Nachteil dieser Methode ist die fehlende Sicht auf die Struktur des Netzwerks. Eine reine Analyse der Kommunikationsdaten gibt keine bis wenig Auskunft über die physischen Gegebenheiten des Netzes.

Ein weiterer Ansatz basiert auf Methoden des maschinellen Lernens und der Clusteranalyse. Hierbei werden Hosts basierend auf ihrer Ähnlichkeit in Gruppen zusammengefasst, ohne zuvor harte Labels zuzuweisen. Dies ist ein praktisches Beispiel für „unsupervised machine learning“. Anstatt zuvor anzugeben: „Dieser Host ist ein Datenbankserver“, findet der Algorithmus selbst ähnliche Hosts anhand ihres Verhaltens und fasst diese in derselben Gruppe zusammen. Für die Bewertung kann eine Vielzahl an Metriken herangezogen werden. Die bekanntesten dabei sind „traffic patterns“, wie die Kennzahlen zu „inbound“ und „outbound traffic“ sowie die verwendeten Protokolle und Ports. Dabei lassen sich beispielsweise Hosts, die ausschliesslich HTTPS-Kommunikation entgegennehmen, als Web-Server gruppieren. Gleiches ist auch über eine Rollenerkennung möglich. Das erfordert meist eine Übersicht der genutzten Ressourcen oder der installierten Software der Hosts. So sind Systeme auch anhand der laufenden Services oder des verwendeten Betriebssystems in Cluster einzuteilen. Neben „traffic patterns“ und „roles“ ist auch „behavior“ ein geeignetes Mittel zur Segmentierung. So können auch Gruppen anhand von Verhaltensähnlichkeiten Sinn ergeben. Haben Hosts vergleichsweise hohe Bandbreite zu definierten Stosszeiten, kann das auf einen ähnlichen Verwendungszweck wie beispielsweise Backup-Systeme hindeuten. Smeriga und Jirsik untersuchen hierzu K-Means und DBSCAN zur Erkennung verhaltenskonsistenter Netzwerksegmente. Die Experimente zeigen, dass aus IP-Flow-Merkmalen relevante Gruppierungen abgeleitet werden können, diese jedoch nicht zwingend mit organisatorisch definierten Segmenten übereinstimmen Smeriga und Jirsik, 2019. Obwohl diese Methoden in der Lage sind, Muster in grossen Datenmengen zu erkennen, weisen sie praktische Nachteile auf. Insbesondere die geringe Interpretierbarkeit der Ergebnisse erschwert deren Einsatz in sicherheitskritischen Umgebungen, da Administratoren die resultierenden Entscheidungen oft nur schwer nachvollziehen können.

Beim dritten Ansatz, dem sogenannten Policy Mining, werden meist bestehende Firewall-Regeln als Grundlage für die Segmentierung des Netzwerks genutzt. Das ist zumeist eine triviale Lösung und bietet sich für eine einfache und rasche Umsetzung an. Da der Ansatz bestehende Konfigurationen nutzt, besteht die Gefahr, dass fehlerhafte oder historisch gewachsene Regelwerke reproduziert werden. Dadurch fließen bestehende Schwächen in die neue Lösung ein und werden nicht behoben.

Die drei genannten Ansätze zeigen, dass unterschiedliche Methoden zur Segmentierung existieren, jedoch jeweils spezifische Einschränkungen aufweisen. Insbesondere fehlt häufig eine konsistente strukturelle Modellierung des Netzwerks, die eine systematische Analyse und Optimierung ermöglicht.

3.2 Graphbasierte Ansätze

Durch die Modellierung des Netzwerks als Graph werden Systeme wie Hosts, virtuelle Maschinen oder Dienste als Knoten dargestellt, während Kommunikationsbeziehungen zwischen solchen Systemen als Kanten abgebildet werden. Diese Art der Modellierung ermöglicht es, auch die physische Struktur des Netzes abzubilden. Im Gegensatz zu den rein datengetriebenen Ansätzen wird nämlich nicht nur die Kommunikation betrachtet, sondern auch die darunterliegende Infrastruktur explizit mit abgebildet. Knoten und Kanten können mit weiteren Attributen versehen werden, etwa Ports, Protokollen oder Kommunikationshäufigkeiten, wodurch ein detailliertes Modell des Netzwerks entsteht. Mhaskar et al. zeigen einen formalen graphbasierten Ansatz, bei dem Ressourcen und Zugriffskontrollpunkte gemeinsam modelliert werden. Aus den Sicherheitsrichtlinien der Ressourcen werden eine robuste Netzwerktopologie und die zugehörigen Firewall-Policies abgeleitet Mhaskar et al., 2021. Rahman und Al-Shaer gehen hingegen von einer bereits bestehenden Topologie aus und synthetisieren geeignete Sicherheitsmassnahmen sowie deren Positionierung innerhalb des Netzwerks Rahman und Al-Shaer, 2013.

Einer der Hauptvorteile dieser Herangehensweise ist die Möglichkeit, etablierte Analyseverfahren aus der Graphentheorie anzuwenden. Darunter sind beispielsweise die Identifikation zentraler Knoten mittels Zentralitätsmassen, die Analyse von Abhängigkeiten entlang von Pfaden oder die Erkennung von Gruppenstrukturen durch Community-Detection-Algorithmen. Dadurch können Zusammenhänge im Netzwerk einfacher erkannt und genauer untersucht werden.

Ein praktisches Beispiel für die Identifikation von eng miteinander kommunizierenden Systemgruppen ist etwa die Kommunikation innerhalb einer mehrschichtigen Architektur wie Web-, Applikations- und Datenbankebene. Solche Gruppen können als Grundlage für Segmentierungsentscheidungen dienen, indem Kommunikation innerhalb der Gruppe erlaubt und zwischen Gruppen gezielt eingeschränkt wird.

Trotz dieser Vorteile weisen auch graphenbasierte Ansätze Einschränkungen auf. Die Abhängigkeit von der Datenqualität ist speziell bei Graphen stark. Fehlende oder fehlerhafte Verbindungen können die Segmentierung erheblich beeinflussen, sodass unvollständige Graphen häufig zu ungenauen Clustern führen. Viele Verfahren benötigen zudem Parameter wie die Anzahl der Cluster, die Gewichtung der Kanten oder Auflösungsparameter. Hinzu kommt, dass gefundene Communities meist mathematisch sinnvoll, aber nicht immer fachlich nachvollziehbar sind. Bei einer über den Graphen generierten Lösung kann unklar sein, warum bestimmte Knoten zusammengefasst wurden und welche Eigenschaften das Segment tatsächlich charakterisieren.

Insgesamt bieten graphbasierte Ansätze eine leistungsfähige Grundlage zur Analyse von Netzwerkstrukturen und zur Identifikation potenzieller Segmentierungsstrategien. Für eine konsistente und überprüfbare Umsetzung von Segmentierungsregeln ist jedoch eine Ergänzung durch formale Verfahren erforderlich.

3.3 Vergleich der Ansätze

Die betrachteten Ansätze zur Netzwerksegmentierung unterscheiden sich deutlich hinsichtlich ihrer Methodik, Aussagekraft und praktischen Anwendbarkeit. Verfahren, die auf der Analyse von Netzwerkverkehr basieren, ermöglichen es, bestehende Kommunikationsbeziehungen sichtbar zu machen. Sie liefern wertvolle Einblicke in tatsächlich genutzte Verbindungen und können als Grundlage für Segmentierungsentscheidungen dienen. Allerdings fehlt diesen Ansätzen eine explizite strukturelle Modellierung des Netzwerks. Insbesondere in komplexen Umgebungen erschwert dies die Ableitung konsistenter und langfristig wartbarer Segmentierungsstrategien.

Ansätze aus dem Bereich des maschinellen Lernens und der Clusteranalyse sind in der Lage, Muster in grossen Datenmengen zu erkennen und Systeme automatisch zu gruppieren. Dennoch ist die praktische Anwendbarkeit häufig eingeschränkt. Die resultierenden Segmentierungen sind oft schwer nachvollziehbar, was insbesondere in sicherheitskritischen Kontexten problematisch ist. Zudem besteht die Herausforderung, geeignete Merkmale und Parameter zu definieren, um sinnvolle Ergebnisse zu erzielen.

Policy-Mining-Ansätze bieten den Vorteil, bestehende Regelwerke wiederzuverwenden. In der Praxis führt dies jedoch häufig dazu, dass bestehende Schwächen oder Inkonsistenzen übernommen werden. Eine echte Optimierung der Segmentierung findet in vielen Fällen nicht statt, da die zugrunde liegenden Probleme nicht grundlegend adressiert werden.

Graphbasierte Ansätze unterscheiden sich von den genannten Methoden dadurch, dass sie das Netzwerk explizit als strukturierte Einheit modellieren. Durch die Kombination aus struktureller Darstellung und analytischen Verfahren ermöglichen sie eine ganzheitliche Betrachtung von Systemen und deren Beziehungen. Insbesondere die Möglichkeit, Abhängigkeiten, Kommunikationspfade und Gruppierungen innerhalb des Netzwerks zu analysieren, stellt einen entscheidenden Vorteil dar.

Im direkten Vergleich zeigt sich, dass graphbasierte Modelle eine geeignete Grundlage für die systematische Analyse komplexer Netzwerke bieten. Sie vereinen die Fähigkeit zur Darstellung von Struktur und Verhalten und ermöglichen gleichzeitig den Einsatz leistungsfähiger Analyseverfahren.

Gleichzeitig wird deutlich, dass graphbasierte Ansätze allein nicht alle Anforderungen vollständig erfüllen. Insbesondere fehlt häufig eine formale Absicherung der abgeleiteten Segmentierungsstrategien. Vor diesem Hintergrund erscheint eine Kombination aus graphbasierter Modellierung und ergänzenden Verfahren zur formalen Überprüfung und Optimierung als besonders vielversprechend.

Die betrachteten Forschungsarbeiten verdeutlichen dabei zwei unterschiedliche Grundrichtungen. Datengetriebene Verfahren leiten potenzielle Segmente aus beobachtetem Kommunikationsverhalten ab Smeriga und Jirsik, 2019, während formale Verfahren explizite Sicherheitsanforderungen und Netzwerktopologien als Grundlage für eine überprüfbare Synthese verwenden Mhaskar et al., 2021; Rahman und Al-Shaer, 2013.

4 Systemdesign

In diesem Kapitel wird das Systemdesign des entwickelten Ansatzes beschrieben. Zunächst werden die Gesamtarchitektur sowie das Constraint- und Netzwerkmodell vorgestellt. Anschließend folgen die Graph Engine, die heuristischen und solver-basierten Segmentierungsansätze, die Gruppenmodellierung sowie der Web-Editor zur Erstellung und Bearbeitung von Netzwerkszenarien.

4.1 Gesamtarchitektur

Das entwickelte System folgt einer modularen Architektur zur automatisierten Mikrosegmentierung von Netzwerken. Ziel ist es, aus abstrakten Kommunikationsanforderungen eine konsistente Segmentierungsempfehlung abzuleiten. Die Verarbeitung erfolgt in einer Pipeline über mehrere Stufen. Als Ausgangspunkt dienen sogenannte High-Level-Constraints, welche gewünschte oder verbotene Kommunikationsbeziehungen zwischen Systemen beschreiben. Diese werden zunächst in ein strukturiertes Constraint-Modell überführt. Die Trennung zwischen Netzwerktopologie, abstrakten Sicherheitsanforderungen und der eigentlichen Synthese findet sich ebenfalls in formalen Security-Design-Frameworks, welche diese Informationen als getrennte Eingaben modellieren und anschließend gemeinsam verarbeiten Rahman und Al-Shaer, 2013.

Mit diesen Informationen als Basis wird eine digitale Kopie des Netzwerks erstellt. Dieses Digital-Twin-Modell beschreibt die vorhandenen Systeme sowie deren Verbindungen und bildet als Graph die Grundlage für die weitere Analyse.

Darauf aufbauend werden zwei unterschiedliche Segmentierungsansätze verfolgt. Zum einen eine heuristische Methode, welche iterativ versucht, eine effiziente Lösung für die definierten Constraints zu finden. Zum anderen wird ein SMT-Solver angewendet, der eine eigenständige Lösung auf Basis der definierten Constraints berechnet. Beide Ansätze werden unabhängig voneinander ausgeführt und erzeugen jeweils eigene Segmentierungsergebnisse. Dies ermöglicht einen direkten Vergleich der Eigenschaften, der Korrektheit, der Struktur und der Skalierbarkeit.

Als Ergebnis wird eine vollständige Segmentierung des gewünschten Netzwerks ausgegeben. Dies erfolgt in Form einer interaktiven Netzwerkansicht, einer VLAN-Zuweisung aller Systeme und einer Ausgabe des definierten Regelwerks. Durch die klare Trennung der einzelnen Arbeitsschritte kann das System effizient mit neuen Erweiterungen ausgestattet werden.

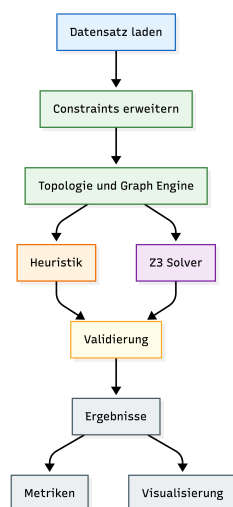


Abbildung 4.1: Visualisierung der Verarbeitungspipeline

4.2 Datenmodelle

4.2.1 Constraint Modell

Das Constraint Modell bildet die formale Grundlage für die Beschreibung des Segmentierungsproblems. Ziel ist es, Kommunikationsanforderungen und Isolationsregeln in einer strukturierten Form abzubilden, sodass diese algorithmisch verarbeitet und überprüft werden können.

Ausgangspunkt des Modells sind sogenannte High-Level Constraints, welche erlaubte (ALLOW) und verbotene (DENY) Kommunikationsbeziehungen zwischen Systemen definieren. Zusätzlich wird die Netzwerktopologie berücksichtigt, um sicherzustellen, dass abgeleitete Lösungen auch physisch realisierbar sind. Die formale Trennung von gewünschten Kommunikationsbeziehungen, Isolationsanforderungen und technischen Entscheidungsvariablen entspricht dem Grundprinzip constraint-basierter Security-Design-Synthese Rahman und Al-Shaer, 2013.

Zur Modellierung der Segmentierung wird jedem Netzwerkknoten eine oder mehrere VLAN-Zugehörigkeiten zugewiesen. Hierzu wird für jeden Knoten n und jedes VLAN v eine boolesche Variable definiert:

$$V(n, v) \in \{0, 1\}$$

Dabei gilt:

$$V(n, v) = 1 \Leftrightarrow n \text{ gehört zu VLAN } v$$

Diese Darstellung ermöglicht es, die Segmentierung als logisches Entscheidungsproblem zu formulieren.

Für Hosts wird eine eindeutige Zuordnung zu genau einem VLAN gefordert:

$$\forall h \in \text{Hosts} : \sum_v V(h, v) = 1$$

Im Gegensatz dazu können Netzwerkgeräte wie Switches mehrere VLANs gleichzeitig tragen:

$$\forall s \in \text{Switches} : \sum_v V(s, v) \geq 1$$

Um sicherzustellen, dass Kommunikationsbeziehungen auch tatsächlich realisierbar sind, wird zusätzlich die Netzwerktopologie berücksichtigt. Ein VLAN muss entlang eines Kommunikationspfades durchgängig existieren:

$$\forall v, \forall n \in \text{path}(A, B) : V(n, v) = 1$$

Die eigentlichen Kommunikationsanforderungen werden über ALLOW- und DENY-Constraints modelliert. Eine erlaubte Kommunikation zwischen zwei Knoten A und B erfordert die Existenz eines gemeinsamen VLANs entlang eines gültigen Pfades:

$$\exists v \ (V(A, v) = 1 \wedge V(B, v) = 1 \wedge \forall n \in \text{path}(A, B) : V(n, v) = 1)$$

Im Gegensatz dazu wird für verbotene Kommunikation gefordert, dass kein solches VLAN existiert:

$$\forall v \ \neg (V(A, v) = 1 \wedge V(B, v) = 1 \wedge \forall n \in \text{path}(A, B) : V(n, v) = 1)$$

Neben einzelnen Knoten können auch Gruppen von Systemen definiert werden, die gemeinsam betrachtet werden. Eine Gruppe wird als Menge von Knoten beschrieben:

$$G = \{n_1, n_2, \dots, n_k\}$$

Constraints können dabei sowohl auf einzelne Knoten als auch auf ganze Gruppen angewendet werden. Beispielsweise wird ein ALLOW-Constraint zwischen zwei Gruppen als Menge aller paarweisen Kommunikationsbeziehungen interpretiert.

Das beschriebene Modell ermöglicht es, Segmentierungsprobleme formal zu definieren und bildet die Grundlage für die Anwendung von Constraint-Solving-Verfahren. Gleichzeitig bleibt es flexibel genug, um unterschiedliche Anforderungen und Netzwerkstrukturen abzubilden.

4.2.2 Netzwerkmodell (Digital Twin)

Das Netzwerkmodell stellt die konkrete Umsetzung des zuvor beschriebenen Constraint-Modells als Digital Twin dar. Ziel ist es, die physische und logische Struktur eines Netzwerks für die algorithmische Verarbeitung optimiert abzubilden. Modelliert wird das Netzwerk durch Knoten und Kanten. Knoten repräsentieren die einzelnen Komponenten und werden in unterschiedliche Typen unterteilt, insbesondere in Hosts und Netzwerkgeräte wie Switches. Diese Unterscheidung ist für die spätere Segmentierung relevant, da unterschiedliche Knotentypen unterschiedlichen Einschränkungen unterliegen, beispielsweise hinsichtlich ihrer VLAN-Zugehörigkeit. Kanten beschreiben die Verbindungen zwischen den Knoten. Sie repräsentieren physische und logische Netzwerkverbindungen und definieren mögliche Kommunikationspfade innerhalb des Netzes. Die explizite Berücksichtigung der Topologie ist notwendig, da die Realisierbarkeit von Sicherheitsmassnahmen nicht allein von den Endpunkten, sondern auch von den möglichen Kommunikationspfaden und den dazwischenliegenden Netzwerkgeräten abhängt Rahman und Al-Shaer, 2013.

Ein wichtiger Teil des Modells ist die Berücksichtigung von Pfaden zwischen Knoten. Kommunikation wird nicht nur auf direkte Verbindungen reduziert, sondern entlang vollständiger Pfade betrachtet. Dadurch wird sichergestellt, dass die abgeleiteten Segmentierungsstrategien plausibel und mit der tatsächlichen Netzwerktopologie konsistent sind.

Zur Veranschaulichung kann das Netzwerkmodell in einer strukturierten Form, beispielsweise als JSON-Dokument, dargestellt werden. Ein vereinfachtes Beispiel ist nachstehend dargestellt.

Listing 1: Beispiel eines einfachen Netzwerkszenarios

```

1 {
2   "nodes": ["User1", "User2", "User3", "User4", "Printer1"],
3   "switches": ["Switch1", "Switch2"],
4
5   "groups": [
6     { "name": "ICT", "members": ["User1", "User2"] },
7     { "name": "Office", "members": ["User3", "User4"] }
8   ],
9
10  "constraints": [
11    { "type": "ALLOW", "src": "User1", "dst": "Printer1" },
12    { "type": "DENY", "src": "ICT", "dst": "Office" },
13
14    { "type": "LOCATED_AT", "node": "User1", "switch": "Switch1" },
15    { "type": "LOCATED_AT", "node": "User2", "switch": "Switch2" },
16    { "type": "LOCATED_AT", "node": "User3", "switch": "Switch1" },
17    { "type": "LOCATED_AT", "node": "User4", "switch": "Switch1" },
18    { "type": "LOCATED_AT", "node": "Printer1", "switch": "Switch2" },
19
20    { "type": "CONNECTED", "node_a": "Switch1", "node_b": "Switch2" }
21  ]
22 }
```

Das Beispiel zeigt ein einfaches Netzwerk, in dem mehrere Hosts über zwei Switches miteinander verbunden sind. Auf Basis dieser Struktur können Kommunikationspfade bestimmt und anschliessend im Constraint-Modell weiterverarbeitet werden.

Das Netzwerkmodell bildet somit die zentrale Grundlage für alle weiteren Verarbeitungsschritte. Es stellt die Verbindung zwischen abstrakten Anforderungen und der realen Netzwerktopologie her.

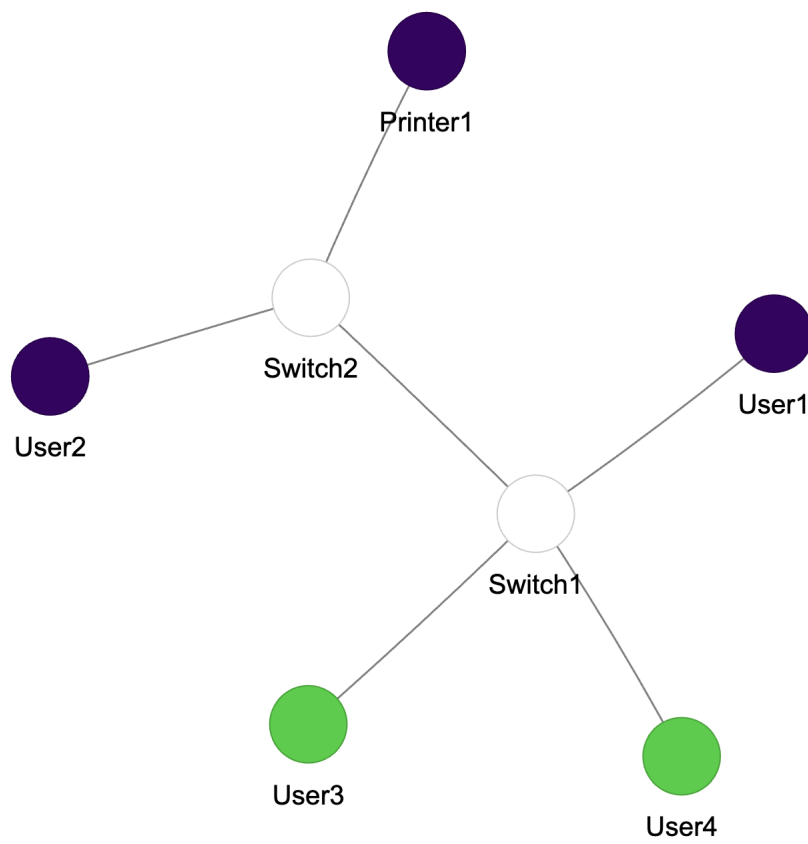


Abbildung 4.2: Einfaches Netzwerkszenario

4.3 Graph Engine

Die Graph Engine bildet die zentrale Verarbeitungsschicht zwischen dem Netzwerkmodell und den darauf aufbauenden Segmentierungsverfahren. Sie stellt grundlegende Funktionen zur Analyse der Netzwerkstruktur zur Verfügung. Insbesondere untersucht sie Kommunikationspfade zwischen Systemen und bestimmt Pfade innerhalb des Netzwerks. Für zwei gegebene Knoten wird berechnet, ob ein gültiger Kommunikationspfad existiert und welche Zwischenknoten dabei durchlaufen werden. In dieser Arbeit wird primär der kürzeste Pfad zwischen zwei Systemen betrachtet, da dieser eine naheliegende Approximation für reale Kommunikationswege darstellt.

Diese Annahme orientiert sich an tatsächlichen Mechanismen in Computernetzwerken. In klassischen Layer-2-Netzwerken werden beispielsweise durch Protokolle wie das Spanning Tree Protocol redundante Pfade reduziert, sodass eine schleifenfreie Baumstruktur entsteht. In Layer-3-Netzwerken bestimmen Routing-Protokolle wie OSPF oder IS-IS den jeweils kürzesten oder kostengünstigsten Pfad, basierend auf definierten Metriken. In beiden Fällen wird effektiv ein optimaler Pfad durch das Netzwerk genutzt. Den kürzesten Pfad im Modell zu verwenden, ist daher eine sinnvolle Vereinfachung, die das Verhalten realer Netzwerke hinreichend gut approximiert, ohne die Komplexität unnötig zu erhöhen.

Neben der Berechnung von Pfaden zwischen Knoten spielt auch die Erreichbarkeitsanalyse eine zentrale Rolle. Für viele Fragestellungen ist es ausreichend, zu bestimmen, ob grundsätzlich eine Verbindung zwischen zwei Systemen möglich ist. Dabei ist es nicht nötig, den genauen Pfad im Detail zu betrachten. Beispielsweise reicht für die Prüfung, ob Kommunikationsanforderungen überhaupt erfüllbar sind, eine Erreichbarkeitsprüfung völlig aus.

Weiter schafft die Graph Engine eine Abstraktion der zugrunde liegenden Graphenrepräsentation. Die eigentliche Datenstruktur sowie die verwendete Bibliothek zur Graphverarbeitung werden gegenüber den darüber liegenden Komponenten gekapselt. Die Implementierung erfolgt über die Bibliothek NetworkX, deren Funktionalität jedoch nicht direkt exponiert wird. Diese Abstraktion bietet mehrere Vorteile. Zum einen wird die Komplexität der verwendeten Bibliothek vom restlichen System entkoppelt, wodurch die Wartbarkeit und Verständlichkeit erhöht werden. Zum anderen ermöglicht sie eine klare Definition von Schnittstellen. Über die ausschliesslich benötigten Operationen wie Pfadberechnung oder Erreichbarkeitsprüfung werden diese bereitgestellt. Der Austausch der zugrunde liegenden NetworkX-Bibliothek bleibt so grundsätzlich möglich, ohne die übrigen Komponenten des Systems anpassen zu müssen.

Die über die Graph Engine ermittelten Pfade werden im weiteren Verlauf durch das Constraint-Modell verwendet. ALLOW- und DENY-Constraints beziehen sich nicht nur auf die Endpunkte einer Kommunikation, sondern auch auf die Existenz eines durchgängigen Pfades im Netzwerk. Die Graph Engine liefert somit die notwendige Grundlage, um solche Constraints korrekt auszuwerten und sicherzustellen, dass abgeleitete Segmentierungsstrategien mit der tatsächlichen Topologie konsistent sind.

4.4 Segmentierungsansätze

4.4.1 Heuristischer Ansatz

Der Ansatz wird als heuristisch bezeichnet, weil keine globale Optimierung stattfindet, sondern die Entscheidungen zu einer gültigen Segmentierung lokal getroffen werden. Im Gegensatz zu dem formal basierten Verfahren steht hierbei nicht eine optimale Lösung im Vordergrund, sondern eine schnell berechenbare Approximation.

Ausgangspunkt sind die ALLOW-Constraints, welche die gewünschten Kommunikationsbeziehungen zwischen den Systemen beschreiben. Für jede Beziehung wird zunächst überprüft, ob im Graphenmodell ein gültiger Pfad existiert. Trifft dies zu, kann der Pfad als Grundlage für die weitere Segmentierung verwendet werden. Die zentrale Idee des Algorithmus besteht darin, entlang dieser Pfade VLAN-Zuweisungen vorzunehmen. Alle ALLOW-Constraints werden schrittweise abgearbeitet und wo möglich einem bestehenden VLAN zugewiesen. Die Auswahl der VLANs erfolgt wiederum unter Berücksichtigung der definierten DENY-Constraints. Bevor ein bestehendes VLAN erneut verwendet wird, wird überprüft, ob dadurch verbotene Kommunikationsbeziehungen entstehen würden. Ist dies der Fall, wird ein neues VLAN erzeugt. Auf diese Weise wird sichergestellt, dass die Segmentierung die definierten Isolationsanforderungen respektiert und konsistent ist.

Der Algorithmus arbeitet iterativ über alle ALLOW-Constraints und baut schrittweise eine vollständige Segmentierung auf. Das Resultat ist dabei von der Reihenfolge der Verarbeitung abhängig und kann im resultierenden Ergebnis je nach Definition des gleichen Netzes variieren.

Insgesamt zeichnet sich der heuristische Ansatz durch seine geringe Komplexität und hohe Ausführungsgeschwindigkeit aus. Gleichzeitig ist zu beachten, dass keine Garantie für eine optimale oder minimale Lösung besteht. Die Qualität der erzeugten Segmentierung hängt von den gegebenen Constraints sowie der Struktur des Netzwerks ab.

4.4.2 Solver-basierter Ansatz (Z3)

Neben dem heuristischen Verfahren wird ein formal basierter Ansatz zur Segmentierung eingesetzt, der auf Constraint Solving unter Verwendung eines SMT-Solvers basiert. Ziel ist es, eine Segmentierung zu berechnen, die alle definierten Anforderungen vollständig erfüllt und konsistent ist. Analog dem heuristischen Verfahren bildet auch hier das im Abschnitt 4.2.1 beschriebene Constraint Model die Basis.

Grundlage bildet das in Abschnitt 4.2.1 beschriebene Constraint Modell. Dabei werden die zu bestimmenden VLAN-Zuweisungen als boolesche Entscheidungsvariablen modelliert. Für jeden Knoten und jedes mögliche VLAN wird eine Variable definiert, die angibt, ob der jeweilige Knoten diesem VLAN zugeordnet ist. Die Segmentierungsaufgabe wird somit in ein logisches Entscheidungsproblem umgewandelt.

Anhand dieser Grundlage werden verschiedene Arten von Constraints formuliert. Zu Beginn wird sichergestellt, dass Hosts nur einem VLAN zugeordnet sind. Netzwerkgeräte hingegen können mehrere VLANs tragen. Weiterhin sind die Kommunikationsanforderungen in Form von ALLOW- und DENY-Constraints abgebildet. Die Constraints-Regeln beinhalten dabei nicht nur die Endpunkte, sondern prüfen zugleich die Verfügbarkeit eines gültigen Pfades innerhalb des Netzwerks. ALLOW-Constraints verlangen, dass für zwei Knoten mindestens ein gemeinsames VLAN existiert, über das eine Kommunikation entlang eines gültigen Pfades möglich ist. Im Gegensatz dazu erzwingen DENY-Constraints, dass kein solches VLAN existieren darf. Durch diese Formulierung wird sowohl Konnektivität als auch Isolation explizit modelliert.

Der Solver verarbeitet die Gesamtheit aller Constraints gleichzeitig und sucht nach einer Belegung der Variablen die alle Bedingungen erfüllt. Die gemeinsame Kodierung von Topologie-, Konnektivitäts- und Isolationsbedingungen als SMT-Problem wurde bereits erfolgreich für die automatisierte Synthese von Netzwerksicherheitskonfigurationen eingesetzt Rahman und Al-Shaer, 2013. Im Gegensatz zum heuristischen Ansatz erfolgt die Lösungssuche somit global und nicht schrittweise. Dies ermöglicht es, auch komplexe Abhängigkeiten zwischen verschiedenen Anforderungen zu berücksichtigen.

Ein wesentlicher Vorteil dieses Ansatzes ist die Fähigkeit zur Erkennung von Konflikten. Falls keine Belegung existiert, die alle Constraints erfüllt, wird das Problem als nicht erfüllbar (UNSAT) klassifiziert. UNSAT-Cores können dabei verwendet werden, um diejenigen Bedingungen einzugrenzen, die gemeinsam zur Nichterfüllbarkeit des Modells führen Rahman und Al-Shaer, 2013. Dies stellt einen wichtigen Mehrwert gegenüber rein heuristischen Verfahren dar, bei denen solche Konflikte häufig unentdeckt bleiben. Im Falle einer erfüllbaren Problemstellung liefert der Solver eine vollständige Segmentierung in Form von VLAN-Zuweisungen für alle Knoten. Diese Lösung erfüllt alle definierten Anforderungen und ist somit formal korrekt im Sinne des zugrunde liegenden Modells.

Gleichzeitig ist zu beachten, dass der solver-basierte Ansatz mit höherem Rechenaufwand verbunden ist und mit zunehmender Netzwerkgröße an Skalierbarkeitsgrenzen stoßen kann. In der vorliegenden Arbeit wird dieser Ansatz daher ergänzend zum heuristischen Verfahren eingesetzt, um dessen Ergebnisse zu vergleichen und die Eigenschaften beider Methoden systematisch zu untersuchen.

4.5 Gruppenmodellierung

Die im Prototyp umgesetzte Gruppenmodellierung erweitert eine Gruppenregel vor der Verarbeitung in einzelne Beziehungen zwischen den beteiligten Hosts. Dieser Ansatz funktioniert für kleine Gruppen, führt bei zunehmender Gruppengröße jedoch zu einer starken Zunahme der erzeugten Constraints. Insbesondere bei paarweisen Beziehungen wächst deren Anzahl annähernd quadratisch, wodurch Laufzeit und Speicherbedarf nicht mehr primär durch das eigentliche Segmentierungsverfahren, sondern durch die vorgelagerte Expansion bestimmt werden.

Aus diesem Grund wurden Gruppen in den Experimenten bewusst nicht verwendet. Andernfalls hätte die aktuelle Implementierung die Ergebnisse der Evaluation erheblich beeinflusst und einen fairen Vergleich zwischen Heuristik und Z3-Solver erschwert. Die Zusammenfassung von Ressourcen mit ähnlichen Eigenschaften oder Sicherheitsrichtlinien zu gemeinsamen Einheiten ist jedoch ein grundlegendes Prinzip formaler Segmentierungsansätze Mhaskar et al., 2021. Die Gruppenmodellierung stellt somit eine konzeptionell sinnvolle Erweiterung dar, wurde im Rahmen des Prototyps jedoch noch nicht ausreichend effizient umgesetzt. Eine weiterführende Lösung müsste Gruppen direkt im Constraint-Modell abbilden, ohne sämtliche Beziehungen vorab einzeln zu expandieren.

4.6 Systemkomponenten (Web-Editor)

Zur Unterstützung der Interaktion mit dem System wurde zudem eine eigenständige Web-Applikation entwickelt. Diese basiert auf einer leichtgewichtigen Architektur unter Verwendung von Python und Flask im Backend sowie HTML, CSS und JavaScript im Frontend. Ziel des Werkzeugs ist es, Netzwerke in strukturierter Form zu erstellen, zu bearbeiten und als JSON zu exportieren.

Innerhalb der Anwendung können verschiedene Elemente eines Netzwerks definiert werden. Dazu zählen Hosts, Switches sowie deren Verbindungen. Darüber hinaus lassen sich Gruppen von Systemen erstellen und Kommunikationsregeln in Form von ALLOW- und DENY-Constraints festlegen. Die resultierende Konfiguration wird kontinuierlich als JSON-Dokument visualisiert, wodurch eine direkte Rückmeldung über die aktuelle Modellstruktur entsteht. Ein besonderer Fokus liegt auf der einfachen und schnellen Erstellung von Netzwerkszenarien. Benutzer können bestehende Konfigurationen importieren, anpassen und erneut exportieren, wodurch sich unterschiedliche Szenarien einfach erstellen und vergleichen lassen. Die erzeugten Daten können anschliessend direkt in das entwickelte Segmentierungssystem übertragen werden.

Insgesamt trägt die Web-Applikation dazu bei die entwickelte Methodik praktisch zugänglich zu machen. Sie erleichtert insbesondere die Erstellung synthetischer Testdaten und unterstützt die Analyse.

5 Implementierung

In diesem Kapitel wird die konkrete Umsetzung des zuvor beschriebenen Systemdesigns erläutert. Ziel ist es, die getroffenen Architekturentscheidungen in ihrer praktischen Realisierung nachzuvollziehen und die wesentlichen Komponenten der Implementierung zu beschreiben.

Zunächst wird die Wahl der verwendeten Technologien begründet und deren Eignung für die Umsetzung des Systems diskutiert. Anschließend wird die Projektstruktur vorgestellt, um einen Überblick über die Organisation des Quellcodes und die Aufteilung in einzelne Module zu geben. Weiter werden die Kernkomponenten beleuchtet und die wesentlichen Entscheide bei der Implementierung ausgebreitet.

5.1 Technologiewahl

Als zentrale Programmiersprache kommt Python zum Einsatz. Python bietet eine hohe Ausdrucksstärke sowie eine grosse Anzahl an verfügbaren Bibliotheken, insbesondere im Bereich der Datenverarbeitung und algorithmischen Analyse. In der Technologieanalyse zum Start des Projektes hob sich Python mit den angebotenen Tools zur Netzwerkanalyse markant von anderen Programmiersprachen ab. Darüber hinaus ermöglicht die klare und kompakte Syntax eine gut nachvollziehbare Implementierung der entwickelten Verfahren.

Für die Modellierung und Analyse der Netzwerkstruktur wird die Bibliothek NetworkX verwendet. Diese stellt eine umfangreiche Funktionalität zur Verarbeitung von Graphen bereit und ermöglicht insbesondere die effiziente Berechnung von Pfaden und Erreichbarkeiten. Gleichzeitig erlaubt sie eine flexible Erweiterung der Graphstruktur durch zusätzliche Attribute, was für die Abbildung von Netzwerkmodellen von Vorteil ist.

Zur Umsetzung des solver-basierten Ansatzes wird der SMT-Solver Z3 eingesetzt. Z3 ermöglicht die Formulierung und Lösung komplexer Constraint-Systeme und bietet Unterstützung für verschiedene Theorien, die für die Modellierung von Segmentierungsproblemen erforderlich sind. Insbesondere erlaubt der Einsatz eines solchen Solvers die formale Überprüfung von Lösungen sowie die Erkennung von widersprüchlichen Anforderungen. Z3 wurde bereits in vergleichbaren Arbeiten eingesetzt, um Netzwerktopologien, Sicherheitsanforderungen und Konnektivitätsbedingungen als gemeinsames SMT-Modell zu verarbeiten Rahman und Al-Shaer, 2013.

Für die Entwicklung des webbasierten Editors wird das Framework Flask verwendet. Flask ermöglicht die Erstellung leichtgewichtiger Webanwendungen mit minimalem Overhead und eignet sich insbesondere für prototypische Anwendungen. Ein weiterer Vorteil besteht darin, dass sowohl der Editor als auch das Segmentierungssystem in derselben Programmiersprache umgesetzt sind. Dadurch kann die Integration beider Komponenten vereinfacht werden, beispielsweise durch gemeinsame Datenstrukturen oder die direkte Weiterverarbeitung erzeugter Netzwerkkonfigurationen.

5.2 Projektstruktur

Auf oberster Ebene enthält das Projekt verschiedene Verzeichnisse zur Organisation von begleitenden Materialien. Dazu zählen insbesondere Dokumentation (**docs**), wissenschaftliche Quellen (**papers**) sowie das eigentliche Thesis-Dokument.

Der zentrale Bestandteil der Implementierung befindet sich im Verzeichnis **src**. Dieses ist in zwei Hauptmodule unterteilt: Das Segmentierungssystem (**microseg**) selbst, sowie die Web-Applikation zur Erstellung von Netzwerken (**network-builder**).

Das Modul **microseg** enthält die Kernlogik des Systems und ist weiter in mehrere Teilbereiche gegliedert. Im Paket **constraints** wird das formale Constraint-Modell definiert, welches die Grundlage für die solver-basierte Segmentierung bildet. Das Modul **topology** beschreibt die Struktur des Netzwerks und bildet die Basis für den Digital Twin. Darauf aufbauend stellt das Modul **graph** die Graph Engine bereit, welche Funktionen zur Pfadberechnung und Erreichbarkeitsanalyse implementiert.

Die eigentliche Segmentierungslogik ist im Modul **synthesis** umgesetzt. Hier werden sowohl der heuristische Ansatz als auch der solver-basierte Ansatz integriert und ausgeführt. Ergänzend dazu dienen die Module **validation** und **visualization** zur Überprüfung sowie zur Aufbereitung der erzeugten Ergebnisse.

Neben dem Segmentierungssystem enthält das Verzeichnis **src** auch das Modul **network-builder**, welches die Web-Applikation zur Erstellung und Bearbeitung von Netzwerkszenarien umfasst. Diese ist getrennt von der Kernlogik organisiert, greift jedoch auf kompatible Datenstrukturen zurück, wodurch eine direkte Integration der erzeugten Konfigurationen ermöglicht wird.

Durch diese Struktur wird eine klare Trennung der Verantwortlichkeiten erreicht. Gleichzeitig bleibt das System flexibel erweiterbar, da einzelne Module unabhängig voneinander angepasst oder ausgetauscht werden können.

5.3 Umsetzung der Kernkomponenten

5.3.1 Graph Builder

Der Graph Builder ist verantwortlich für die Erstellung des Netzwerkmodells auf Basis der definierten Constraints. Ziel ist es, aus einer abstrakten Beschreibung von Systemen und deren Beziehungen eine konkrete, verarbeitbare Graphstruktur zu erzeugen.

Die grundlegenden Bausteine des Modells sind Knoten und Kanten. Knoten repräsentieren die einzelnen Systeme im Netzwerk und werden durch einen eindeutigen Identifier sowie einen Typ beschrieben. Dabei wird zwischen Hosts und Switches unterschieden:

Listing 2: Definition der Knoten und Knotentypen

```
1 class NodeType(str, Enum):
2     HOST = "host"
3     SWITCH = "switch"
4
5
6 @dataclass
7 class Node:
8     id: str
9     type: NodeType
```

Kanten modellieren die Verbindungen zwischen diesen Systemen und werden durch die beteiligten Knoten definiert:

Listing 3: Definition der Kanten

```
1 @dataclass
2 class Edge:
3     node_a: str
4     node_b: str
```

Zur Speicherung und Verarbeitung der Topologie wird eine Graphstruktur verwendet, die auf der Bibliothek NetworkX basiert. Diese wird durch eine eigene Abstraktion gekapselt, um eine klare Trennung zwischen Modell und Implementierung zu gewährleisten:

Listing 4: Implementierung der Graphstruktur

```
1 class TopologyGraph:
2     def __init__(self):
3         """Wrap a NetworkX graph for topology storage."""
4         self.graph = nx.Graph()
```

Der Aufbau des Graphen erfolgt schrittweise durch die Auswertung der gegebenen Constraints. Zunächst werden alle Switches identifiziert und als Knoten im Graphen angelegt. Diese Information wird aus den Verbindungs-Constraints zwischen Netzwerkgeräten abgeleitet. Anschliessend werden Hosts hinzugefügt und über entsprechende Zuordnungen mit den jeweiligen Switches verbunden. Darauf aufbauend werden die noch ausstehenden Verbindungen zwischen den Switches angelegt, wodurch die vollständige Netzwerktopologie entsteht.

Durch diese schrittweise Konstruktion wird sichergestellt, dass sowohl die physische Struktur des Netzwerks als auch die Beziehungen zwischen den einzelnen Komponenten korrekt abgebildet werden. Der Graph Builder stellt somit die Grundlage für alle weiteren Verarbeitungsschritte dar, da er die initiale Struktur erzeugt, auf der sowohl die Graph Engine als auch die Segmentierungsverfahren aufbauen.

5.3.2 Graph Engine

Die durch den Graph Builder erzeugte Topologie stellt die Grundlage für die weitere Analyse der Netzwerkstruktur dar. Dieses Modell wird direkt an die Graph Engine übergeben, welche darauf aufbauend Funktionen zur Pfadberechnung und Erreichbarkeitsanalyse bereitstellt.

Die Graph Engine wird dabei mit der vollständigen Topologie initialisiert und greift auf die zugrunde liegende Graphstruktur zu:

Listing 5: Initialisierung der Graph Engine

```
1 class PathEngine:
2     """Initialize with a topology; stores underlying NetworkX graph."""
3
4     def __init__(self, topology: TopologyGraph):
5         self.graph = topology.graph
```

Auf Basis dieses Graphen können zentrale Operationen durchgeführt werden. Dazu zählt insbesondere die Bestimmung des kürzesten Pfades zwischen zwei Knoten:

Listing 6: Bestimmung des kürzesten Pfades zwischen zwei Knoten

```
1 path = nx.shortest_path(self.graph, source=src, target=dst)
```

Ein zentraler Aspekt der Graph Engine ist die Abstraktion der verwendeten Graphbibliothek. Die konkrete Implementierung auf Basis von NetworkX wird bewusst gekapselt und nicht direkt in den darüberliegenden Komponenten verwendet. Stattdessen stellt die Graph Engine eine klar definierte Schnittstelle für benötigte Operationen wie Pfadberechnung oder Erreichbarkeitsprüfung bereit.

Durch die Trennung zwischen Graph Builder und Graph Engine entsteht somit eine lose Kopplung zwischen Modellierung und Analyse. Während der Graph Builder für die Konstruktion der Topologie verantwortlich ist, übernimmt die Graph Engine deren Auswertung. Die von der Graph Engine bereitgestellten Informationen werden anschließend sowohl im heuristischen als auch im solver-basierten Ansatz verwendet, insbesondere zur Auswertung von ALLOW- und DENY-Constraints entlang gültiger Kommunikationspfade.

5.3.3 Heuristic Solver

Die Umsetzung des heuristischen Segmentierungsansatzes ist in mehrere Schritte zur Verarbeitung der definierten Kommunikationsanforderungen gegliedert. Der Fokus liegt hierbei auf einer effizienten Verarbeitung anhand lokaler Entscheidungen entlang von Kommunikationspfaden. Im Gegensatz zu einem globalen Optimierungsansatz wird dabei bewusst auf eine inkrementelle Konstruktion gesetzt. Dies reduziert die Komplexität der Lösung und skaliert unter einigen Voraussetzungen besser.

Zentrale Datenstruktur für die Speicherung der Ergebnisse ist das Objekt `SegmentationResult`. Dieses verwaltet die Zuordnung zwischen Knoten und VLANs in beide Richtungen:

Listing 7: Datenstruktur zur Speicherung der Segmentierungsergebnisse

```
1 class SegmentationResult:
2     def __init__(self):
3         """
4         Store bidirectional mappings: node to VLANs and VLAN to nodes.
5         Uses defaultdicts for convenient incremental updates.
6         """
7
8         self.node_to_vlans = defaultdict(set)
9         self.vlan_to_nodes = defaultdict(list)
```

Die bidirektionale Struktur ermöglicht sowohl eine effiziente Abfrage der VLAN-Zugehörigkeit einzelner Knoten. Dadurch wird die spätere Analyse der erzeugten Segmentierung insbesondere im Hinblick auf Konsistenzprüfungen deutlich vereinfacht.

Der eigentliche Algorithmus ist im `HeuristicSynthesizer` implementiert. Diese Klasse verarbeitet iterativ alle `ALLOW`-Constraints und berechnet für jede Kommunikationsanforderung einen Pfad im Netzwerk:

Listing 8: Verarbeitung der `ALLOW`-Constraints im `HeuristicSynthesizer`

```
1 path = self.path_engine.get_path(src, dst)
```

Bei der Verarbeitung der Constraints ist die Prüfung eines gültigen Pfades über die Path Engine ein zentrales Element. Gibt es keinen passenden Kommunikationspfad, kann die Anforderung verworfen werden, da keine konsistente Segmentierung möglich ist. Existiert ein gültiger Pfad, wird entlang dieses Pfades eine VLAN-Zuweisung vorgenommen. Dabei wird versucht, ein bereits existierendes VLAN wiederzuverwenden. Hierzu werden alle VLANs betrachtet, die bereits einem der Knoten entlang des Pfades zugeordnet sind. Damit reduziert sich die Anzahl benötigter VLANs und fördert eine kompakte Struktur. Eine Wiederverwendung ist nur möglich, wenn dadurch keine `DENY`-Constraints verletzt werden.

Kann keines der bereits bestehenden VLAN verwendet werden, wird ein neues VLAN erzeugt. Dieses wird anschliessend allen Knoten entlang des Pfades zugewiesen. Bei diesem Ablauf entsteht schrittweise eine vollständige Segmentierung des Netzwerks. Bei der Verarbeitung jedes `ALLOW`-Constraints kann die bestehende Struktur erweitert oder angepasst werden.

Bei der Zuweisung der Hosts wird jeweils geprüft, dass die maximale Anzahl von VLANs für den Systemtyp nicht überschritten wird. Müsste ein Host anhand der Kommunikationsanforderungen in zwei VLAN sein, führt dieser Widerspruch automatisch zu einem Abbruch des Segmentierungsvorgangs.

Die Prüfung von `DENY`-Constraints erfolgt dynamisch während der VLAN-Auswahl. Dabei wird überprüft, ob die Zuweisung eines VLANs entlang eines Pfades zu einer verbotenen Kommunikationsbeziehung mit bereits bestehenden Zuordnungen führen würde. Dies verhindert, dass inkonsistente Segmentierungen entstehen und stellt sicher, dass alle negativen Sicherheitsanforderungen jederzeit eingehalten werden.

5.3.4 Z3 Solver

Der Z3 Solver Ansatz basiert auf der Abbildung des Segmentierungsproblems auf ein formales Constraint-System. Ziel ist es, eine globale Lösung zu berechnen, die alle definierten Anforderungen gleichzeitig erfüllt.

Zur Implementierung sind boolesche Entscheidungsvariablen zu definieren, die die Zugehörigkeit eines Knotens zu einem VLAN beschreiben. Für jede Kombination aus Knoten und VLAN wird eine variable erzeugt:

Listing 9: Definition der booleschen Entscheidungsvariablen

```
1 # V(node, vlan) to Bool
2 self.V = {
3     (node, vlan): Bool(f"V_{node}_{vlan}") for node in nodes for vlan in vlans
4 }
```

Diese Variablen bilden die direkte Umsetzung des in Abschnitt 4.2.1 beschriebenen Modells und stellen die Grundlage für alle weiteren Constraints dar. Die weitere Modellierung erfolgt durch das schrittweise Hinzufügen von Constraints zum Solver. Zunächst werden strukturelle Einschränkungen wie die Anzahl möglicher VLANs pro Host definiert. Dies ist über eine Kardinalitätsbedingung realisierbar, bei der genau eine der zugehörigen Variablen (VLANs) den Wert 'wahr' annehmen darf.

Aufbauend werden die Kommunikationsanforderungen modelliert. ALLOW-Constraints werden so formuliert, dass mindestens ein VLAN existieren muss, über das zwei Knoten entlang eines gültigen Pfades miteinander verbunden sind. Dies wird durch eine Disjunktion über alle möglichen VLAN-Zuweisungen realisiert.

Gehen wir von einem Beispielpfad *HostA* - *Switch1* - *Switch2* - *HostB* aus und den möglichen VLANs 10, 20 und 30. So entstehen die logischen Verbindungen von A nach B über die dazwischen liegenden Switches mit jedem VLAN. Eine Lösung ergibt sich bei der Sammlung aller Conditions und der Bildung einer Disjunktion über den 'Or' Operator.

Listing 10: Erzeugung der Bedingungen für die möglichen VLANs

```

1 vlan_conditions = []
2
3 for v in self.vlans:
4     cond = And(
5         self.V[(c.src, v)],
6         self.V[(c.dst, v)],
7         *[self.V[(node, v)] for node in path],
8     )
9     vlan_conditions.append(cond)

```

$$C_{10} = V(A, 10) \wedge V(B, 10) \wedge V(S1, 10) \wedge V(S2, 10)$$

$$C_{20} = V(A, 20) \wedge V(B, 20) \wedge V(S1, 20) \wedge V(S2, 20)$$

$$C_{30} = V(A, 30) \wedge V(B, 30) \wedge V(S1, 30) \wedge V(S2, 30)$$

Listing 11: Verknüpfung der VLAN-Bedingungen durch eine Disjunktion

```

1 constraint = Or(vlan_conditions)

```

$$C_{10} \vee C_{20} \vee C_{30}$$

DENY-Constraints werden hingegen restriktiv formuliert. Für jedes VLAN wird ausgeschlossen, dass beide Endpunkte sowie alle Knoten entlang eines Pfades gleichzeitig diesem VLAN angehören. Dadurch wird sichergestellt, dass verbotene Kommunikationsbeziehungen in keiner möglichen Konfiguration auftreten können.

Bei den Berechnungen der Constraints wird die Prüfung eines gültigen Pfades zwischen den Knoten bereits integriert. So löst die Graph Engine beim übersetzen der Constraints die möglichen Pfade zwischen den betrachteten Hosts dynamisch auf. Dadurch wird sichergestellt, dass die Topologie des Netzwerks vollständig in die Entscheidungsfindung einbezogen wird.

Der wesentliche Vorteil gegenüber dem heuristischen Ansatz ist die globale Betrachtung des kompletten Constraints-Systems. Alle Anforderungen werden gleichzeitig betrachtet, wodurch auch komplexe Abhängigkeiten korrekt berücksichtigt werden können. Existiert eine gültige Lösung, wird das Modell ausgelesen und in eine strukturierte Segmentierung überführt. Weiter kann der Solver eine sogenannte UNSAT-Core liefern, die eine Teilmenge widersprüchlicher Constraints identifiziert. Diese Informationen werden in der Implementierung gruppiert und auf die ursprünglichen Anforderungen zurückgeführt. Dadurch kann gezielt analysiert werden, welche Kommunikationsanforderungen eine konsistente Lösung verunmöglichen. Ein ähnlicher Mechanismus wird in ConfigSynth verwendet, um bei einem nicht erfüllbaren Modell widersprüchliche Schwellenwerte und Anforderungen zu identifizieren und alternative erfüllbare Konfigurationen zu bestimmen Rahman und Al-Shaer, 2013.

Zusammenfassend stellt der solver-basierte Ansatz eine formale und vollständige Methode zur Segmentierung dar. Er ermöglicht die Berechnung konsistenter Lösungen sowie die Erkennung von Konflikten, ist jedoch mit einem höheren Rechenaufwand verbunden als der heuristische Ansatz.

5.3.5 Datenimport / Export

Für die Eingabe von Netzwerken wird ein JSON-basiertes Format verwendet. Dieses erlaubt eine klare und flexible Beschreibung der relevanten Elemente, insbesondere von Knoten, Verbindungen sowie Kommunikationsanforderungen. Beim Import werden diese Daten in die internen Datenstrukturen des Systems überführt und anschließend für die Erstellung des Netzwerkmodells sowie die Segmentierung verwendet.

Die Ausgabe der Ergebnisse erfolgt auf zwei unterschiedlichen Ebenen. Zum einen wird in der Kommandozeile eine strukturierte Darstellung des gesamten Verarbeitungsprozesses ausgegeben. Diese umfasst unter anderem die Anzahl geladener Constraints, die Größe der erzeugten Topologie sowie die Ausführungszeiten der einzelnen Segmentierungsverfahren. Zusätzlich werden die resultierenden VLAN-Zuweisungen für beide Ansätze in textueller Form dargestellt, wodurch ein direkter Vergleich der Ergebnisse möglich ist.

Listing 12: Strukturierte Ausgabe des Verarbeitungsprozesses und der Segmentierungsergebnisse

```
1  === LOAD ===
2  Constraints loaded: 8
3
4  === EXPANSION ===
5  Expanded constraints: 13
6
7  === TOPOLOGY ===
8  Nodes: 7
9  Edges: 6
10
11 === HEURISTIC ===
12 Execution Time: 0.000272 seconds
13 Status: SUCCESS
14
15 === Z3 SOLVER ===
16 Status: SAT
17 Execution Time: 0.011567 seconds
18
19 === RESULT ===
20 Heuristic Solution:
21 VLAN 10: ['Printer1', 'Switch1', 'Switch2', 'User1', 'User2']
22 VLAN 20: ['Switch1', 'User3', 'User4']
23
24 Z3 Solution:
25 VLAN 10: ['Printer1', 'Switch1', 'Switch2', 'User1', 'User2']
26 VLAN 20: ['Switch1', 'User3', 'User4']
27
28 === VISUALIZATION ===
29 Rendering Heuristic...
30 filepath heuristic.html
31 Rendering Z3...
32 filepath z3_solver.html
33
34 === PERFORMANCE ===
35 Heuristic Time: 0.000272 seconds
36 Z3 Time:      0.011567 seconds
37
38 Z3 / Heuristic Ratio: 42.55x
```

Zum anderen erfolgt eine visuelle Aufbereitung der Ergebnisse in Form interaktiver HTML-Dokumente. Hierbei wird das Netzwerk graphisch dargestellt und die berechnete Segmentierung durch farbliche Markierung der VLANs visualisiert. Diese Darstellung ermöglicht es, auf intuitive Weise zu analysieren, welche Knoten zu welchen Segmenten gehören und wie die Kommunikation innerhalb des Netzwerks strukturiert ist.

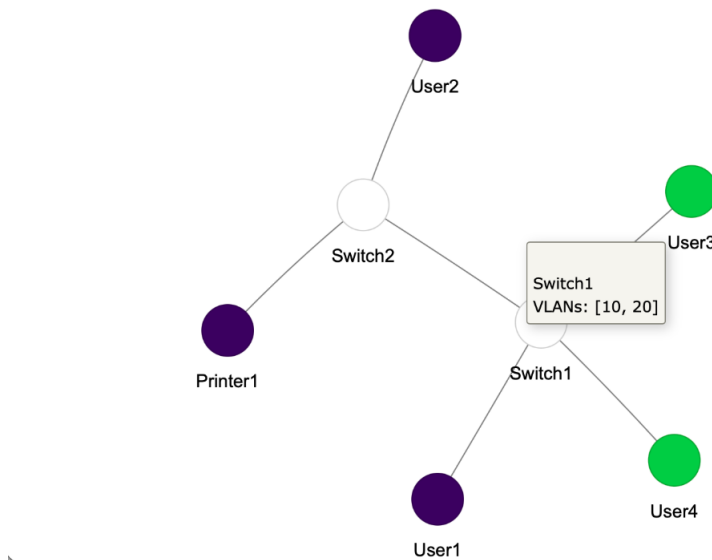


Abbildung 5.1: Einfaches Beispiel der Graphenvisualisierung

Insgesamt trägt die Import- und Exportfunktionalität dazu bei, das Segmentierungstool erfahrbar und einfacher nutzbar zu machen.

6 Evaluation

In diesem Kapitel wird das entwickelte System anhand definierter Experimente untersucht und bewertet. Ziel ist es, die Eigenschaften der implementierten Segmentierungsansätze zu analysieren und deren Verhalten unter unterschiedlichen Bedingungen zu vergleichen.

6.1 Ziel der Evaluation

Im Mittelpunkt steht die Untersuchung zentraler Eigenschaften wie Laufzeit, Skalierbarkeit und Qualität der erzeugten Segmentierungen. Speziell wird analysiert, wie sich beide Ansätze bei unterschiedlich grossen Netzwerken verhalten und welche Unterschiede sich in Bezug auf die Anzahl der verwendeten Segmente sowie die Einhaltung der definierten Constraints ergeben.

Ein weiterer Fokus liegt auf der Bewertung der jeweiligen Stärken und Schwächen. Während der heuristische Ansatz auf eine effiziente und schnelle Berechnung abzielt, bietet der solver-basierte Ansatz eine formale Absicherung der Ergebnisse. Die Evaluation soll aufzeigen, in welchen Szenarien welcher Ansatz Vorteile bietet und wie sich beide Verfahren im direkten Vergleich verhalten.

6.2 Versuchsaufbau

Zur Evaluation des entwickelten Systems wurden synthetische Netzwerkszenarien mit unterschiedlichen Eigenschaften erzeugt. Ziel ist es, das Verhalten der beiden Segmentierungsansätze unter kontrollierten Bedingungen systematisch zu untersuchen.

Die Datensätze basieren auf drei grundlegenden Topologien:

- **Star** - ein zentraler Switch mit direkt angebundenen Hosts als einfache Baseline
- **Tree** - eine hierarchische Struktur mit Core-, Distribution- und Access-Switches als realitätsnahe Abbildung von Unternehmensnetzwerken
- **Mesh** - ein stark vernetztes Switch-Netzwerk mit hoher Pfadvielfalt und erhöhter Komplexität

Für diese drei Topologietypen wurden verschiedene Netzwerke in unterschiedlichen Grössen erzeugt. Dabei werden kleine bis mittlere Netzwerke von 5 bis 120 Hosts als eigene Szenarien abgedeckt. Zusätzlich wird die Komplexität der Kommunikationsanforderungen über die Constraints-Dichte gesteuert. Hierbei werden drei Stufen unterschieden:

- **Low** - wenige Kommunikationsregeln
- **Medium** - mittlere Anzahl von Regeln
- **High** - hohe Anzahl an Kommunikationsanforderungen und Einschränkungen

Eine mögliche Steigerung der Komplexität ist in der nachstehenden Tabelle abgebildet:

Listing 13: Beispielhafte Anzahl der Constraints in den Experimenten

1	----- ----- ----- ----- -----
2	dataset hosts allow_cons. deny_cons. total_cons.
3	----- ----- ----- ----- -----
4	star_005_low.json 5 3 0 3
5	----- ----- ----- ----- -----
6	star_005_medium.json 5 4 1 5
7	----- ----- ----- ----- -----
8	star_005_high.json 5 4 4 8
9	----- ----- ----- ----- -----

Aus den drei Topologien (Star, Tree, Mesh) werden acht Abstufungen an Hosts geprüft (5, 10, 20, 40, 60, 80, 100, 120), die wiederum in den drei Komplexitätsstufen (Low, Medium, High) abgebildet werden. Daraus ergeben sich 72 Experimente, die automatisiert über ein dediziertes Skript verarbeitet werden können. Für jeden Durchlauf werden sowohl Logdateien als auch strukturierte Messergebnisse erzeugt und in einer zentralen CSV-Datei gespeichert.

Damit der Fokus des Vergleichs in der Evaluation auf die beiden Verfahren gerichtet werden kann und nicht die eigentliche Lösbarkeit betrachtet wird, sind alle Experimente so gestaltet, dass sicher eine mögliche Segmentierung besteht. Die kontrollierte Variation synthetischer Netzwerke ist auch bei vergleichbaren SMT-basierten Syntheseverfahren üblich, um den Einfluss von Netzwerkgrösse, Topologie und Anzahl der Bedingungen isoliert untersuchen zu können Rahman und Al-Shaer, 2013.

Diese Ergebnisse bilden die Grundlage für die anschließende Analyse und die Erstellung der Diagramme.

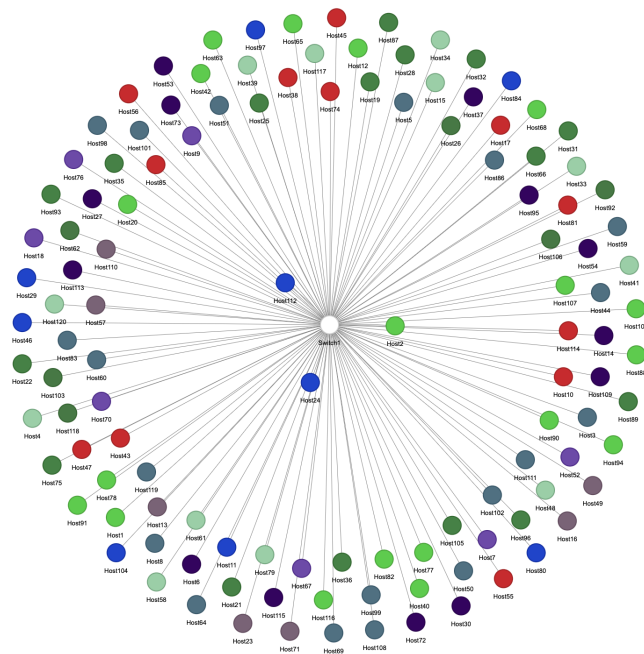


Abbildung 6.1: Star-Topologie mit 120 Hosts

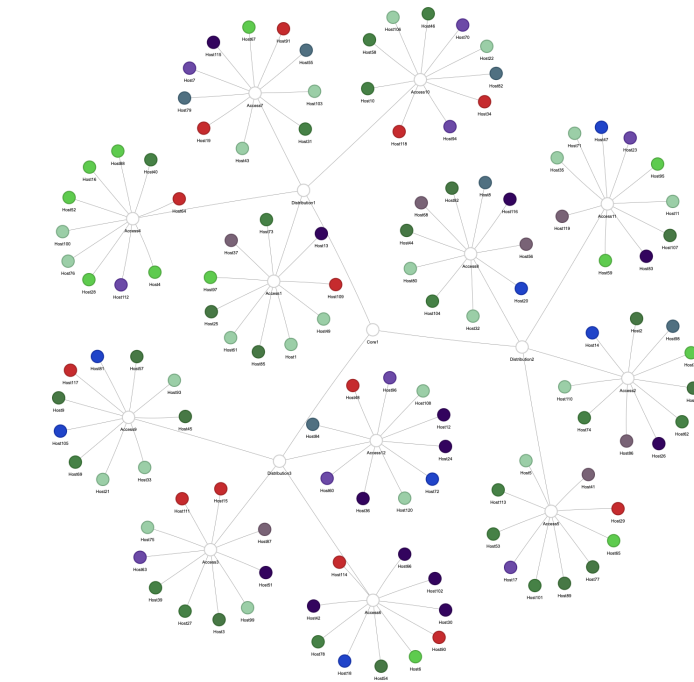


Abbildung 6.2: Tree-Topologie mit 120 Hosts

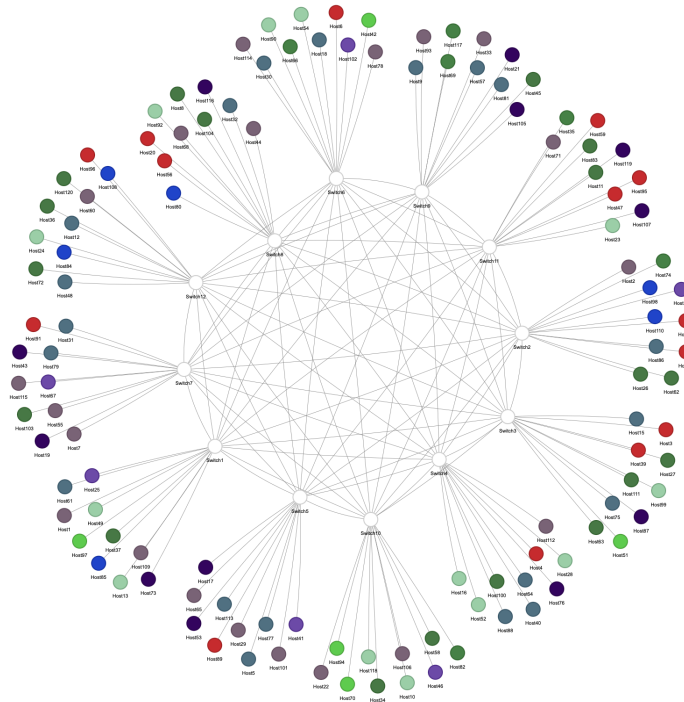


Abbildung 6.3: Mesh-Topologie mit 120 Hosts

6.3 Metriken

Zur Bewertung der beiden Segmentierungsansätze werden durch die Ausführung von Experimenten mehrere Metriken zur Untersuchung der verschiedenen Aspekte herangezogen. Der Fokus liegt dabei auf der Laufzeit der Algorithmen, der Segmentierungsqualität sowie der Korrektheit der ausgegebenen Lösung.

Eine zentrale Metrik in diesem Projekt ist die Laufzeit, welche die benötigte Zeit zur Berechnung einer vollständigen Segmentierung beschreibt. Dieser Aspekt kann insbesondere zur Beurteilung der Skalierbarkeit bei zunehmender Netzwerkgrösse oder Constraint-Dichte verwendet werden.

Um die Segmentierungsqualität zu analysieren, kann die Anzahl der verwendeten VLANs betrachtet werden. Eine geringere Anzahl von VLANs deutet auf eine kompaktere und effizientere Segmentierung hin. Wobei eine höhere Anzahl auf ein stärker fragmentiertes Netzwerk hinweist.

Darüber hinaus wird die Korrektheit der Ergebnisse überprüft. Hierzu wird analysiert, ob die erzeugte Segmentierung alle definierten ALLOW- und DENY-Constraints erfüllt. Diese Validierung erfolgt automatisiert im Anschluss an die Berechnung.

Die erhobenen Metriken werden für jeden Datensatz erfasst und in strukturierter Form gespeichert. Diese Daten dienen als Grundlage für die anschliessende Auswertung und Visualisierung.

Ein verkürzter Auszug der generierten Ergebnisse ist in der nachfolgenden Tabelle abgebildet:

Listing 14: Verkürzter Auszug der erfassten Experimentergebnisse

1	-----	-----	-----	-----	-----	
2	dataset	heuristic_time	heuristic_vlans	z3_time	z3_vlans	..
3	-----	-----	-----	-----	-----	
4	star_005_high.json	0.0002...	2	0.019...	2	..
5	-----	-----	-----	-----	-----	
6	star_005_medium.json	0.0002...	2	0.017...	2	..
7	-----	-----	-----	-----	-----	
8	star_005_low.json	0.0002...	1	0.015...	2	..
9	-----	-----	-----	-----	-----	

6.4 Ergebnisse

Zur Evaluation der beiden implementierten Ansätze, des heuristischen Verfahrens und des Z3-basierten SAT-Solver-Ansatzes, wurden 72 synthetische Testszenarien untersucht. Diese ergeben sich aus der Kombination von drei Netzwerktopologien (Star, Tree, Mesh), acht Netzwerkgrößen (5–120 Hosts) und drei Constraint-Dichten (Low, Medium, High). Für jedes Szenario existiert per Konstruktion eine gültige (SAT) Segmentierung, sodass Abweichungen zwischen den Verfahren ausschließlich auf deren Lösungsverhalten zurückzuführen sind und nicht auf unlösbare Instanzen.

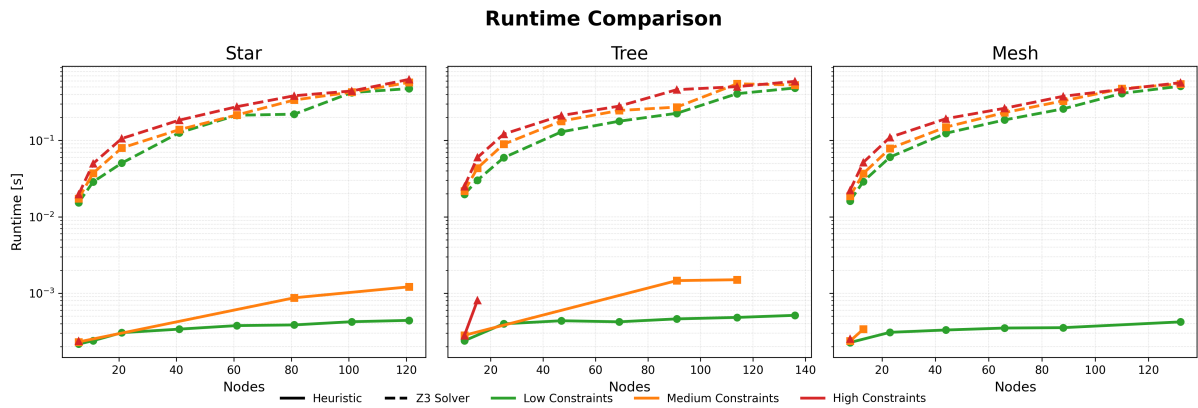


Abbildung 6.4: Laufzeitvergleich der Segmentierungsansätze

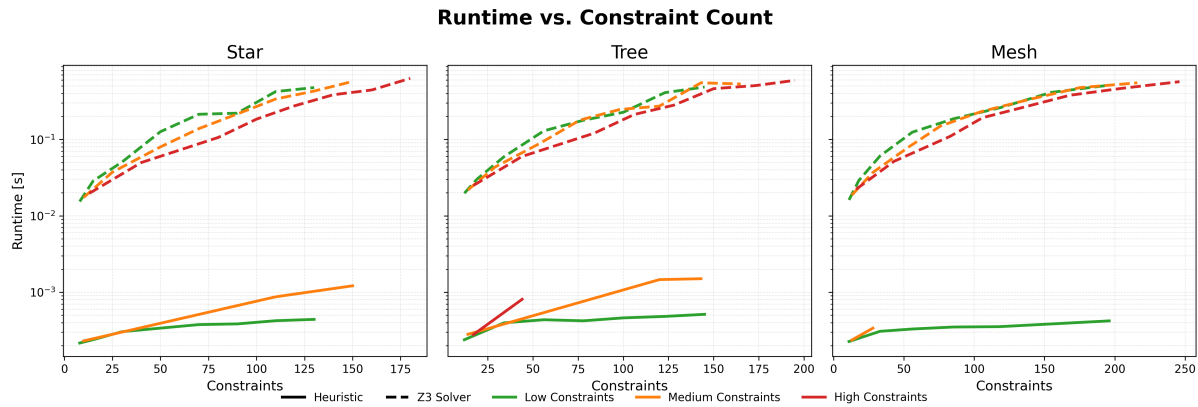


Abbildung 6.5: Einfluss der Anzahl Constraints auf die Laufzeit

Der Z3-Solver zeigt in allen drei Topologien einen deutlichen, nahezu monoton steigenden Laufzeitverlauf mit zunehmender Netzwerkgröße bzw. Constraint-Anzahl. Die Laufzeit steigt von ca. 0,015–0,02 s bei den kleinsten Instanzen auf bis zu 0,55–0,6 s bei den größten Instanzen (bis zu 132 Knoten bzw. 246 Constraints im Mesh-Fall). Die heuristische Lösung liegt dagegen durchgehend im Submillisekundenbereich (ca. 0,0002–0,0006 s) und wächst über den gesamten untersuchten Größenbereich nur marginal. Der resultierende Geschwindigkeitsvorteil (Spaltenratio in den Rohdaten) beträgt im Mittel das 376-fache, mit einer Spannweite von etwa dem 71-fachen bis zum 1219-fachen, wobei die höchsten Beschleunigungsfaktoren bei großen, gering eingeschränkten Mesh-Netzwerken auftreten. Zwischen den Topologien ist der mittlere Beschleunigungsfaktor mit 415 (Star), 378 (Mesh) und 336 (Tree) vergleichbar.

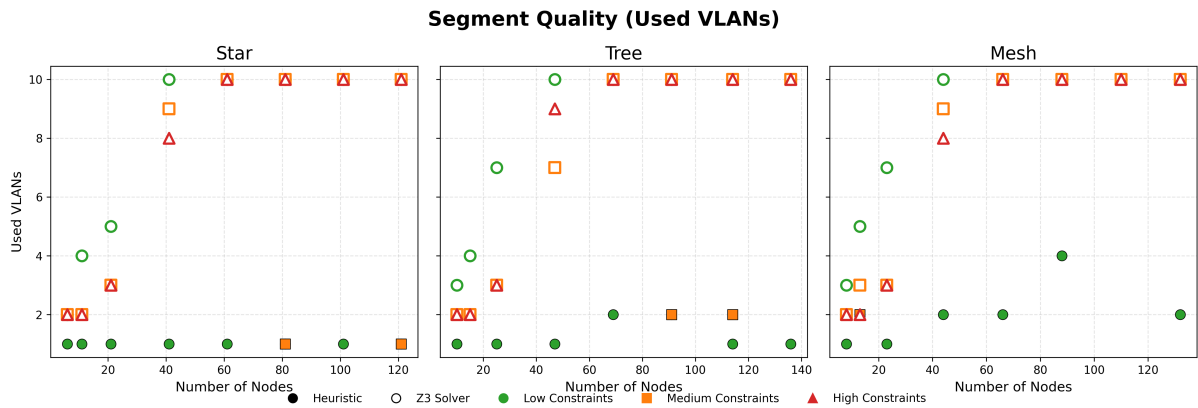


Abbildung 6.6: Vergleich der verwendeten VLANs

Die Heuristik verwendet in nahezu allen Szenarien, in denen sie eine gültige Lösung liefert, eine minimale Anzahl an VLANs. Der Z3-Solver nutzt demgegenüber deutlich mehr Segmente: Der Mittelwert liegt bei 6,9–7,3 VLANs, wobei die Anzahl mit wachsender Netzwerkgröße zunimmt und ab etwa 60 Knoten unabhängig von der Constraint-Dichte bei 10 VLANs sättigt. Bei kleinen Netzwerken (< 25 Knoten) fällt zudem auf, dass die vom Solver genutzte VLAN-Zahl mit der Constraint-Dichte steigt (z. B. Star bei 40 Knoten: 8 VLANs bei High, 9 bei Medium, 10 bei Low).

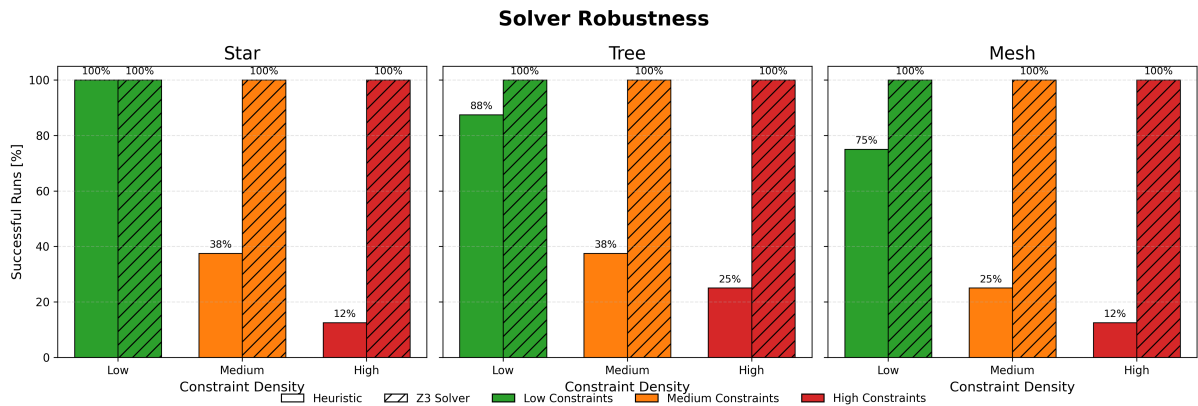


Abbildung 6.7: Vergleich der Robustheit der Segmentierungsansätze

Der Z3-Solver findet in allen 72 Konfigurationen (100 %) eine gültige Segmentierung, was angesichts der garantiert erfüllbaren Datensätze zu erwarten war. Die Erfolgsquote der Heuristik fällt hingegen deutlich ab und ist stark von der Constraint-Dichte abhängig: Bei niedriger Dichte liegt sie bei 100 % (Star), 87,5 % (Tree) bzw. 75 % (Mesh); bei mittlerer Dichte fällt sie auf 37,5 % (Star, Tree) bzw. 25 % (Mesh); bei hoher Dichte auf 12,5 % (Star, Mesh) bzw. 25 % (Tree). Eine detaillierte Betrachtung nach Netzwerkgröße zeigt, dass die Heuristik bei niedriger Dichte über nahezu den gesamten Größenbereich (bis 136 Knoten) erfolgreich bleibt, bei mittlerer und hoher Dichte jedoch bereits ab ca. 15–20 Knoten nahezu durchgängig scheitert.

6.5 Analyse

Die Ergebnisse zeigen einen deutlichen Unterschied bei der Robustheit. Der Z3-Solver erzeugte für alle 72 lösbaren Szenarien eine gültige Segmentierung. Die Heuristik war dagegen nur in 33 Fällen erfolgreich. Besonders bei mittlerer und hoher Constraint-Dichte nahm ihre Erfolgsrate stark ab. Dies deutet darauf hin, dass die Anzahl und Kombination der Kommunikationsregeln einen grösseren Einfluss besitzt als die reine Netzwerkgrösse.

Die Ursache liegt in den unterschiedlichen Lösungsstrategien. Z3 betrachtet alle Bedingungen gleichzeitig und sucht nach einer global gültigen Belegung. Die Heuristik verarbeitet die ALLOW-Anforderungen schrittweise. Früh getroffene VLAN-Zuweisungen können dadurch spätere Entscheidungen blockieren, obwohl grundsätzlich eine gültige Gesamtlösung existiert.

Bei erfolgreichen Durchläufen war die Heuristik deutlich schneller als Z3 und benötigte meist nur wenige Millisekunden. Der Solver zeigte jedoch ebenfalls ein kontrolliertes Laufzeitwachstum und blieb selbst bei den grössten untersuchten Szenarien unter einer Sekunde. Ein Anstieg der Solver-Laufzeit mit wachsender Anzahl von Hosts, Kommunikationsflüssen und Constraints wurde auch in vergleichbaren SMT-basierten Syntheseverfahren beobachtet Rahman und Al-Shaer, 2013.

Die geringere VLAN-Anzahl der Heuristik darf nicht unmittelbar als bessere Lösungsqualität interpretiert werden. Der Z3-Solver sucht derzeit lediglich eine erfüllende Belegung und minimiert die Anzahl verwendeter VLANs nicht. Ein aussagekräftiger Qualitätsvergleich würde daher ein zusätzliches Optimierungsziel im Solver erfordern.

6.6 Diskussion der Ergebnisse

Die Evaluation zeigt, dass der solver-basierte Ansatz insbesondere bei komplexeren Constraint-Sets deutlich zuverlässiger arbeitet. Seine vollständige Erfolgsrate bei allen untersuchten SAT-Instanzen bestätigt, dass die formale Modellierung innerhalb des betrachteten Systems konsistent umgesetzt wurde. Die Heuristik bietet zwar erhebliche Geschwindigkeitsvorteile, verliert jedoch mit steigender Constraint-Dichte deutlich an Robustheit.

Für einfache Szenarien kann die Heuristik dennoch sinnvoll eingesetzt werden, beispielsweise als schnelle Vorberechnung oder als erster Lösungsversuch. Aufgrund ihrer schrittweisen Verarbeitung besteht jedoch das Risiko, dass frühe VLAN-Zuweisungen spätere Anforderungen blockieren. Für einen produktiven Einsatz wäre deshalb eine konsequente Validierung der erzeugten Segmentierung notwendig. Bei einem ungültigen Ergebnis könnte anschliessend automatisch auf den Z3-Solver zurückgegriffen werden.

Die vergleichsweise hohe Anzahl verwendeter VLANs beim Solver ist nicht grundsätzlich als Nachteil des solver-basierten Ansatzes zu bewerten. In der aktuellen Implementierung sucht Z3 lediglich nach einer gültigen Belegung. Die Minimierung der VLAN-Anzahl ist nicht als Optimierungsziel definiert. Eine entsprechende Erweiterung würde einen aussagekräftigeren Vergleich der Segmentierungsqualität ermöglichen und könnte zugleich die praktische Verwendbarkeit der Solver-Lösungen verbessern.

Die Aussagekraft der Ergebnisse ist durch einzelne Messungen pro Datensatz sowie die ausschliessliche Verwendung synthetischer SAT-Szenarien begrenzt. Wiederholte Durchläufe würden zufällige Laufzeiteinflüsse reduzieren und eine Berechnung von Median und Streuung erlauben.

Innerhalb des untersuchten Rahmens ergibt sich dennoch ein klares Bild: Die Heuristik ist im Erfolgsfall wesentlich schneller und erzeugt häufig kompakte Segmentierungen. Z3 benötigt mehr Rechenzeit, liefert dafür jedoch wesentlich zuverlässiger eine gültige Lösung. Für komplexe oder sicherheitskritische Szenarien stellt der solver-basierte Ansatz daher die geeignetere Grundlage dar.

7 Diskussion

In diesem Kapitel werden der entwickelte Ansatz und die Ergebnisse der Evaluation kritisch eingeordnet. Dabei wird betrachtet, welche Vorteile sich aus der Kombination eines graphbasierten Digital Twins mit High-Level-Constraints und automatisierter Regelsynthese ergeben. Gleichzeitig werden die Grenzen des entwickelten Modells sowie technische und methodische Schwierigkeiten aufgezeigt, die während der Umsetzung und Evaluation sichtbar wurden.

7.1 Stärken des Ansatzes

Eine wesentliche Stärke des entwickelten Ansatzes ist die Trennung zwischen den Kommunikationsanforderungen und ihrer technischen Umsetzung. Eine solche deklarative Beschreibung trennt den gewünschten Sicherheitszustand von der konkreten technischen Konfiguration und bildet damit eine zentrale Grundlage automatisierter Security-Design-Synthese Rahman und Al-Shaer, 2013. Erlaubte und verbotene Verbindungen werden als High-Level-Constraints formuliert, ohne dass bereits konkrete VLAN-Zuweisungen vorausgesetzt werden. Dadurch wird deklarativ der gewünschte Zustand des Netzwerks beschrieben, während die eigentliche Segmentierung automatisiert abgeleitet wird.

Das Netzwerk wird als graphbasierter Digital Twin mit Hosts, Switches und physischen Verbindungen modelliert. Im Gegensatz zu einer ausschliesslichen Betrachtung der Endsysteme berücksichtigt das System damit auch die Zwischenknoten eines Kommunikationspfades. Eine ALLOW-Beziehung ist nur dann erfüllt, wenn ein gemeinsames VLAN entlang des gesamten betrachteten Pfades vorhanden ist. DENY-Constraints verhindern entsprechend einen durchgängigen gemeinsamen VLAN-Pfad. Dadurch verbindet das Modell logische Kommunikationsanforderungen mit der vorhandenen Topologie. Die Kombination aus expliziter Topologie und formal beschriebenen Sicherheitsanforderungen ermöglicht es, nicht nur einzelne Endpunkte, sondern auch die technische Realisierbarkeit der resultierenden Sicherheitskonfiguration zu berücksichtigen Rahman und Al-Shaer, 2013.

Positiv ist zudem die Kombination zweier unabhängiger Lösungsverfahren. Die Heuristik stellt einen schnellen, leicht nachvollziehbaren Ansatz dar, während der Z3-Solver nach einer global erfüllenden Belegung sucht. Die Evaluation zeigt, dass die beiden Verfahren unterschiedliche Stärken besitzen: Die Heuristik erreicht bei erfolgreichen Durchläufen sehr kurze Laufzeiten, während der Solver auch bei komplexeren Constraint-Kombinationen zuverlässiger gültige Lösungen erzeugt. Durch die anschliessende Validierung werden die Resultate beider Verfahren anhand derselben Bedingungen überprüft.

Eine weitere Stärke ist die modulare und reproduzierbare Umsetzung. Das Laden und Erweitern der Constraints, der Aufbau des Digital Twins, die Pfadanalyse, die Synthese, die Validierung und die Visualisierung sind als getrennte Komponenten realisiert. Die Experimente können automatisiert ausgeführt werden; Logdateien und Metriken werden strukturiert gespeichert und anschliessend für die Diagrammerstellung verwendet. Dadurch können neue Datensätze und zusätzliche Auswertungen integriert werden, ohne die gesamte Verarbeitungspipeline anzupassen.

7.2 Limitationen

Der entwickelte Digital Twin bildet nur einen vereinfachten und statischen Ausschnitt eines realen Netzwerks ab. Berücksichtigt werden Hosts, Switches und deren Verbindungen. Gerade in dynamischen Netzwerken, in denen Ressourcen und Richtlinien regelmässig verändert werden, sind adaptive Verfahren erforderlich, welche die Segmentierung und Konfiguration kontinuierlich an den aktuellen Zustand anpassen Alabbad, 2021. Dynamische Routingentscheidungen, unterschiedliche Verbindungskosten, redundante Übertragungswege, Ausfälle oder zeitlich veränderliche Kommunikationsbeziehungen sind nicht Bestandteil des Modells. Die Verwendung eines einzelnen kürzesten Pfades stellt deshalb eine Modellannahme dar und entspricht nicht in jedem Fall dem tatsächlich verwendeten Kommunikationsweg.

Auch die Segmentierung selbst ist auf VLAN-Zuweisungen beschränkt. Moderne Microsegmentierung kann zusätzlich durch Firewalls, Access Control Lists, Security Groups, Host-Firewalls oder identitätsbasierte Richtlinien umgesetzt werden. Formale Security-Design-Ansätze berücksichtigen neben vollständiger Zugriffssperre beispielsweise auch vertrauenswürdige Kommunikation über IPSec, Payload-Inspektion durch IDS, NAT und Proxy-Systeme als unterschiedliche Isolationsmuster Rahman und Al-Shaer, 2013. Protokolle, Ports, Applikationen und Benutzeridentitäten werden im aktuellen Modell nicht berücksichtigt. Eine gültige VLAN-Segmentierung entspricht daher nicht automatisch einer vollständigen Microsegmentierungsstrategie für eine produktive Umgebung.

Der Z3-Solver sucht in der aktuellen Implementierung nach einer erfüllenden Belegung, optimiert diese jedoch nicht. Insbesondere wird die Anzahl genutzter VLANs nicht minimiert. Der Solver kann deshalb eine formal gültige, aber administrativ unnötig komplexe Lösung zurückgeben. Das formale Modell sieht eine solche Optimierung grundsätzlich als Erweiterungsmöglichkeit vor.

Die Aussagekraft der Evaluation ist ebenfalls begrenzt. Die Experimente basieren auf synthetisch erzeugten Star-, Tree- und Mesh-Netzwerken mit bis zu 120 Hosts sowie drei definierten Constraint-Dichten. Sämtliche ausgewerteten Datensätze wurden als lösbare SAT-Instanzen erzeugt. Reale Unternehmensnetzwerke, umfangreichere Problemgrößen und systematisch erzeugte UNSAT-Szenarien wurden damit nicht vollständig untersucht.

7.3 Herausforderungen

Eine Herausforderung bestand in der formalen Übersetzung abstrakter Kommunikationsanforderungen in technisch überprüfbare Bedingungen. ALLOW- und DENY-Constraints müssen nicht nur für die jeweiligen Endpunkte, sondern auch für sämtliche Knoten des betrachteten Pfades korrekt abgebildet werden. Gruppen erhöhen die Komplexität zusätzlich, da eine einzelne Gruppenregel in zahlreiche konkrete Beziehungen zwischen Hosts erweitert werden kann.

Schwierig ist ebenfalls das Verhältnis zwischen Ausdrucksstärke und Berechenbarkeit. Eine größere VLAN-Domain und zusätzliche Entscheidungsvariablen erhöhen die Flexibilität des Modells, vergrößern jedoch gleichzeitig den Suchraum des Solvers. Umgekehrt kann eine zu kleine Domain dazu führen, dass ein grundsätzlich segmentierbares Netzwerk innerhalb des vorgegebenen Modells als unlösbar erscheint. Die Wahl geeigneter Grenzen ist daher sowohl eine technische als auch eine methodische Entscheidung.

Bei der Heuristik besteht die Herausforderung darin, mit lokalen Entscheidungen eine global gültige Segmentierung zu erzeugen. Bereits vergebene VLANs beeinflussen spätere Zuordnungen, ohne dass die Heuristik frühere Entscheidungen zurücknehmen kann. Dadurch können lösbare Szenarien scheitern. Eine mögliche Weiterentwicklung wäre ein Backtracking-Verfahren, die Prüfung mehrerer Kandidaten oder eine hybride Strategie, bei der zunächst die Heuristik ausgeführt und bei einem ungültigen Ergebnis automatisch auf den Solver zurückgegriffen wird.

Für eine praktische Anwendung müsste ausserdem sichergestellt werden, dass der Digital Twin die reale Infrastruktur korrekt und aktuell abbildet. Topologieinformationen und Kommunikationsanforderungen müssten aus bestehenden Systemen importiert und die berechneten Ergebnisse in konkrete Konfigurationen verschiedener Hersteller übersetzt werden. Vor einer produktiven Aktivierung wären zusätzliche Prüfungen, Simulationen und kontrollierte Rollout-Mechanismen erforderlich, da fehlerhafte Segmentierungsregeln sowohl Sicherheitslücken, als auch unbeabsichtigte Kommunikationsunterbrüche verursachen können. Für dynamische Umgebungen müsste das Modell Änderungen an Ressourcen, Topologie und Policies automatisch erfassen und eine erneute Segmentierung oder Rekonfiguration auslösen Alabbad, 2021.

8 Fazit und Ausblick

Dieses Kapitel fasst die Ergebnisse der Arbeit zusammen und ordnet sie in Bezug auf die Forschungsfrage ein. Abschliessend werden mögliche Weiterentwicklungen des Prototyps sowie Ansätze für weiterführende Untersuchungen aufgezeigt.

8.1 Zusammenfassung

Ziel dieser Arbeit war die Entwicklung eines Ansatzes, mit dem aus abstrakten Kommunikationsanforderungen automatisiert eine VLAN-basierte Microsegmentierung abgeleitet werden kann. Als Grundlage dient ein graphbasierter Digital Twin, der Hosts, Switches und deren Verbindungen abbildet. Die gewünschten Kommunikationsbeziehungen werden durch High-Level-Constraints wie ALLOW und DENY beschrieben und mit den relevanten Pfaden innerhalb der Netzwerktopologie verknüpft.

Für die Berechnung der Segmentierung wurden zwei voneinander unabhängige Verfahren implementiert. Die Heuristik verarbeitet die Kommunikationsanforderungen schrittweise und weist den beteiligten Netzwerkknoten geeignete VLANs zu. Der Z3-basierte Ansatz überführt das Segmentierungsproblem hingegen in ein formales SMT-Modell und sucht nach einer VLAN-Belegung, die sämtliche modellierten Bedingungen gleichzeitig erfüllt.

Die Evaluation umfasste 72 synthetisch erzeugte Szenarien mit Star-, Tree- und Mesh-Topologien, unterschiedlichen Netzwerkgrössen sowie drei Constraint-Dichten. Die Durchführung der Experimente, die Protokollierung der Ergebnisse und die Erstellung der Diagramme wurden automatisiert.

Die Resultate zeigen einen klaren Zielkonflikt zwischen Geschwindigkeit und Zuverlässigkeit. Die Heuristik benötigte bei erfolgreichen Durchläufen nur sehr wenig Rechenzeit, erzeugte jedoch lediglich für 33 der 72 lösbaren Szenarien eine gültige Segmentierung. Der Z3-Solver fand dagegen für alle untersuchten SAT-Instanzen eine gültige Lösung. Ohne zusätzliches Optimierungsziel verwendete er dabei jedoch häufig mehr VLANs als die Heuristik.

8.2 Beantwortung der Forschungsfrage

Die Forschungsfrage untersucht, wie auf Basis eines graphbasierten Digital Twins und abstrakter Kommunikationsanforderungen automatisiert eine gültige Microsegmentierungsstrategie abgeleitet werden kann.

Die Arbeit zeigt, dass dies durch die Kombination eines Topologiemodells, einer graphbasierten Pfadanalyse und formal definierter High-Level-Constraints möglich ist. Die daraus entstehenden Bedingungen können als Constraint-Satisfaction-Problem formuliert und durch einen SMT-Solver verarbeitet werden. Für sämtliche untersuchten lösbaren Szenarien konnte der Z3-basierte Ansatz eine gültige VLAN-Belegung berechnen.

Die Forschungsfrage kann somit positiv beantwortet werden: Eine automatisierte Mikrosegmentierung lässt sich aus einem Digital Twin und High-Level-Constraints ableiten, sofern die relevanten Netzwerkeigenschaften und Kommunikationsanforderungen ausreichend und vollständig modelliert sind. Die Evaluation zeigt zugleich, dass ein global arbeitender Solver zuverlässiger ist als eine rein schrittweise Heuristik.

8.3 Ausblick

Der entwickelte Prototyp bietet mehrere Möglichkeiten für eine Weiterentwicklung. Ein erster Ansatzpunkt ist die Optimierung des Z3-Modells. Der Solver sucht derzeit lediglich nach einer erfüllenden VLAN-Belegung. Die Anzahl der verwendeten VLANs oder die Komplexität der entstehenden Segmentierung wird nicht minimiert. Durch den Einsatz eines Optimierungsmodells könnte beispielsweise die Anzahl aktiver VLANs reduziert werden. Weitere mögliche Ziele wären eine möglichst geringe Anzahl zusätzlicher VLAN-Zuweisungen auf Switches oder eine gleichmässige Verteilung der Hosts auf die erzeugten Segmente.

Auch die Heuristik kann erweitert werden. Ihre schrittweise Verarbeitung führt dazu, dass frühe Entscheidungen spätere Anforderungen blockieren können. Eine Verbesserung wäre die Bewertung mehrerer VLAN-Kandidaten, bevor eine endgültige Zuweisung vorgenommen wird. Zusätzlich könnten Backtracking, eine Priorisierung besonders restriktiver Constraints oder unterschiedliche Verarbeitungsreihenfolgen untersucht werden. Eine weitere Möglichkeit besteht in einem hybriden Verfahren: Zunächst wird die schnelle Heuristik ausgeführt und validiert. Kann sie keine gültige Segmentierung erzeugen, übernimmt automatisch der Z3-Solver.

Für eine aussagekräftigere Bewertung sollten die Experimente künftig mehrfach ausgeführt werden. Dadurch könnten Median, Mittelwert und Streuung der Laufzeiten bestimmt und zufällige Einflüsse der Ausführungsumgebung reduziert werden. Ergänzend wären grössere Netzwerke sowie systematisch erzeugte UNSAT-Szenarien sinnvoll. Letztere würden nicht nur die Erkennung widersprüchlicher Anforderungen untersuchen, sondern auch zeigen, wie hilfreich die durch den Solver ausgegebenen Konfliktursachen für die Fehleranalyse sind.

Eine weitere Einschränkung liegt im vereinfachten Netzwerkmodell. Der aktuelle Digital Twin bildet Hosts, Switches und statische Verbindungen ab. Für eine realitätsnähere Anwendung könnten zusätzlich Link-Kosten, redundante Pfade, Routinginformationen, Protokolle, Ports und Dienste berücksichtigt werden. Dadurch liessen sich Kommunikationsanforderungen genauer formulieren als durch eine ausschliessliche Betrachtung der beteiligten Endsysteme.

Langfristig wäre zudem eine Integration in bestehende Netzwerkmanagement- und Inventarsysteme denkbar. Eine weiterführende Umsetzung könnte Topologie- und Policy-Änderungen kontinuierlich erfassen und die Segmentierung adaptiv neu berechnen. Vergleichbare Mechanismen wurden insbesondere für die dynamische Konfiguration und Governance von Software-Defined Networks untersucht Alabbad, 2021. Vor einer produktiven Umsetzung wären jedoch zusätzliche Validierungen, Simulationen und kontrollierte Rollout-Mechanismen erforderlich. Zusätzlich könnten Kommunikationsanforderungen künftig teilweise aus beobachteten IP-Flows abgeleitet werden. Aus Merkmalen wie Kommunikationspartnern, Ports, Protokollen und zeitlichen Traffic-Mustern lassen sich Verhaltensprofile und potenzielle Segmentzugehörigkeiten bestimmen Smeriga und Jirsik, 2019. Das Modell könnte zudem um weitere Isolationsmechanismen sowie Optimierungsziele wie Nutzbarkeit, Bereitstellungskosten oder die Platzierung von Sicherheitskomponenten erweitert werden Rahman und Al-Shaer, 2013. Der entwickelte Ansatz bildet dafür eine technische und konzeptionelle Grundlage, auf der sowohl die formale Modellierung als auch die praktische Automatisierung weitergeführt werden können.

Quellenverzeichnis

- Alabbad, M. (2021, Februar). *A formal approach to secure the segmentation and configuration of dynamic networks* [Diss., McMaster University].
- Mhaskar, N., Alabbad, M., & Khedri, R. (2021). A Formal Approach to Network Segmentation. *Computers & Security*, 103, 102162. <https://doi.org/10.1016/j.cose.2020.102162>
- Rahman, M. A., & Al-Shaer, E. (2013). A Formal Framework for Network Security Design Synthesis. *2013 IEEE 33rd International Conference on Distributed Computing Systems*, 560–570. <https://doi.org/10.1109/ICDCS.2013.70>
- Smeriga, J., & Jirsik, T. (2019). Behavior-Aware Network Segmentation using IP Flows. *Proceedings of the 14th International Conference on Availability, Reliability and Security*, 1–9. <https://doi.org/10.1145/3339252.3339265>

9 Glossar

Glossar

ALLOW-Constraint Ein *ALLOW-Constraint* beschreibt eine Kommunikationsanforderung, nach der zwei Systeme über mindestens ein gemeinsames VLAN entlang des betrachteten Netzwerkpades kommunizieren können müssen. 43

Constraint Ein *Constraint* ist eine formal definierte Bedingung, welche die möglichen Lösungen des Segmentierungsproblems einschränkt. 43

Constraint-Dichte Die *Constraint-Dichte* beschreibt die Anzahl der definierten ALLOW- und DENY-Constraints innerhalb eines Netzwerkszenarios. In der Evaluation werden die Stufen Low, Medium und High verwendet. 43

Constraint-Explosion Eine *Constraint-Explosion* bezeichnet die starke Zunahme einzelner Constraints, beispielsweise wenn Gruppenregeln in alle paarweisen Beziehungen ihrer Mitglieder aufgelöst werden. 43

DENY-Constraint Ein *DENY-Constraint* beschreibt eine Isolationsanforderung, nach der zwischen zwei Systemen kein durchgängiges gemeinsames VLAN entlang des betrachteten Netzwerkpades existieren darf. 43

Digital Twin Ein *Digital Twin* ist ein digitales Modell eines realen oder synthetischen Systems. In dieser Arbeit bildet er Hosts, Switches und deren Verbindungen als Graph ab. 43

Graph Ein *Graph* ist eine mathematische Struktur aus Knoten und Kanten, mit der Komponenten und Verbindungen eines Netzwerks dargestellt werden. 43

Graph Builder Der *Graph Builder* erzeugt aus den eingelesenen Topologie-Constraints eine verarbeitbare Graphstruktur. 43

Graph Engine Die *Graph Engine* stellt Funktionen zur Analyse des Netzwerkgraphen bereit, insbesondere zur Pfadberechnung und Erreichbarkeitsprüfung. 43

Heuristik Eine *Heuristik* ist ein Lösungsverfahren, das anhand lokaler und schrittweiser Entscheidungen eine Segmentierung erzeugt. Eine global optimale oder gültige Lösung ist dabei nicht garantiert. 43

High-Level-Constraint Ein *High-Level-Constraint* beschreibt eine abstrakte Kommunikations- oder Isolationsanforderung, ohne bereits eine konkrete technische VLAN-Zuweisung festzulegen. 43

Host Ein *Host* ist ein Endsystem innerhalb des Netzwerkmodells. Im entwickelten Modell wird jeder Host genau einem VLAN zugeordnet. 43

Kante Eine *Kante* verbindet zwei Knoten eines Graphen und repräsentiert eine physische oder logische Verbindung innerhalb der Netzwerktopologie. 43

Knoten Ein *Knoten* repräsentiert eine einzelne Komponente innerhalb eines Graphen. Im Netzwerkmodell werden insbesondere Hosts und Switches als Knoten dargestellt. 43

Microsegmentierung Die *Microsegmentierung* bezeichnet die feingranulare Unterteilung eines Netzwerks, bei der Kommunikationsbeziehungen zwischen einzelnen Systemen oder kleinen Systemgruppen gezielt gesteuert werden. 43

Netzwerktopologie Die *Netzwerktopologie* beschreibt die Struktur eines Netzwerks sowie die Verbindungen zwischen Hosts und Netzwerkgeräten. 43

- Pfad** Ein *Pfad* ist eine Folge miteinander verbundener Knoten zwischen zwei Endpunkten. Die Segmentierung berücksichtigt dabei auch die dazwischenliegenden Switches. 43
- SAT** *SAT* bezeichnet den Zustand eines Constraint-Systems, für das mindestens eine Belegung existiert, welche sämtliche definierten Bedingungen erfüllt. 43
- Segmentierung** Die *Segmentierung* bezeichnet die logische Unterteilung eines Netzwerks in voneinander getrennte Bereiche. In dieser Arbeit erfolgt sie durch VLAN-Zuweisungen. 43
- SMT** *Satisfiability Modulo Theories* erweitert das SAT-Problem um zusätzliche logische und mathematische Theorien zur Modellierung komplexer Entscheidungsprobleme. 43
- Solver** Ein *Solver* ist eine Softwarekomponente, die nach einer Variablenbelegung sucht, welche alle Bedingungen eines formalen Constraint-Systems erfüllt. 43
- Switch** Ein *Switch* ist ein Netzwerkgerät, das Hosts und weitere Switches miteinander verbindet. Im entwickelten Modell kann ein Switch mehrere VLANs gleichzeitig tragen. 43
- UNSAT** *UNSAT* bezeichnet den Zustand eines Constraint-Systems, für das keine Belegung existiert, welche alle definierten Bedingungen gleichzeitig erfüllt. 43
- UNSAT Core** Ein *UNSAT Core* ist eine Teilmenge der Constraints, die gemeinsam für die Nichterfüllbarkeit eines Constraint-Systems verantwortlich ist. 43
- VLAN** Ein *Virtual Local Area Network* ist ein logisches Netzwerksegment, mit dem Systeme unabhängig von der physischen Infrastruktur getrennt oder zusammengefasst werden können. 43
- VLAN-Domain** Die *VLAN-Domain* bezeichnet die Menge der VLANs, die dem Solver für die Berechnung einer Segmentierung zur Verfügung stehen. 43
- Z3** *Z3* ist ein SMT-Solver von Microsoft Research, der in dieser Arbeit zur formalen Berechnung der VLAN-Zuweisungen eingesetzt wird. 43

Eigenständigkeitserklärung

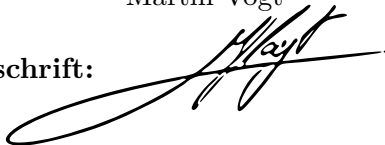
Ich erkläre hiermit, dass ich den vorliegenden Leistungsnachweis selber und selbstständig verfasst habe,

- dass ich sämtliche nicht von mir selber stammenden Textstellen und anderen Quellen wie Bilder etc. gemäss gängigen wissenschaftlichen Zitierregeln¹ korrekt zitiert und die verwendeten Quellen klar sichtbar ausgewiesen habe(n);
- dass ich in einer Fussnote oder einem Hilfsmittelverzeichnis alle verwendeten Hilfsmittel (KI-Assistenzsysteme wie Chatbots², Übersetzungs-³ Paraphrasier-⁴ oder Programmierapplikationen⁵) deklariert und ihre Verwendung bei den entsprechenden Textstellen angegeben habe(n);
- dass ich sämtliche immateriellen Rechte an von mir allfällig verwendeten Materialien wie Bilder oder Grafiken erworben habe(n) oder dass diese Materialien von mir selbst erstellt wurde(n);
- dass das Thema, die Arbeit oder Teile davon nicht bei einem Leistungsnachweis eines anderen Moduls verwendet wurden, sofern dies nicht ausdrücklich mit der Dozentin oder dem Dozenten im Voraus vereinbart wurde und in der Arbeit ausgewiesen wird;
- dass ich mir bewusst bin (sind), dass meine Arbeit auf Plagiate und auf Drittautorschaft menschlichen oder technischen Ursprungs (Künstliche Intelligenz) überprüft werden kann;
- dass ich mir bewusst bin, dass die Hochschule für Technik einen Verstoss gegen diese Eigenständigkeitserklärung bzw. die ihr zugrundeliegenden Studierendenpflichten der Studien- und Prüfungsordnung der Hochschule für Technik verfolgt und dass daraus disziplinarische Folgen (Verweis oder Ausschluss aus dem Studiengang) resultieren können.

Windisch, 5. August 2026

Name: Martin Vogt

Unterschrift:



¹z.B. APA oder IEEE

²z.B. ChatGPT

³z.B. Deepl

⁴z.B. Quillbot

⁵z.B. Github Copilot

Hilfsmittelverzeichnis

Im Rahmen dieser Arbeit wurden die folgenden KI-Assistenzsysteme als unterstützende Hilfsmittel eingesetzt.

ChatGPT-5.6 Sol

ChatGPT-5.6 Sol wurde hauptsächlich bei der Programmierung des entwickelten Prototyps eingesetzt. Die Unterstützung umfasste insbesondere die Ideenfindung, die Diskussion möglicher Lösungsansätze, die Fehlersuche sowie die Erstellung und Überarbeitung einzelner Codeabschnitte. Zusätzlich wurde das System punktuell für die Formulierung und Überarbeitung von Textpassagen verwendet.

Overleaf AI

Overleaf AI wurde vorwiegend für die Rechtschreibkontrolle, die Verbesserung des Satzbaus und die allgemeine sprachliche Prüfung der Arbeit eingesetzt.

Alle durch die KI-Systeme erzeugten Vorschläge wurden vor ihrer Übernahme eigenständig geprüft, angepasst und in den Kontext der Arbeit eingeordnet. Die fachliche Verantwortung für die Inhalte, die Implementierung und die Schlussfolgerungen liegt vollständig beim Autor.

A Anhang

Der Anhang enthält ergänzende Dokumente, die den organisatorischen und methodischen Verlauf der Bachelorarbeit dokumentieren. Dazu gehören die während der Projektlaufzeit erstellten Meetingprotokolle sowie die zu Beginn des Projekts festgelegte Projektvereinbarung. Die Dokumente werden in chronologischer Reihenfolge aufgeführt.

Verzeichnis der Anhänge

Projektvereinbarung	48
Meetingprotokoll vom 25. März 2026	55
Meetingprotokoll vom 7. April 2026	57
Meetingprotokoll vom 21. April 2026	59
Meetingprotokoll vom 12. Mai 2026	62
Meetingprotokoll vom 7. Juli 2026	65
Meetingprotokoll vom 4. August 2026	67

Projektvereinbarung

Automatisierte Microsegmentierungsstrategie basierend auf Digital Twin Netzwerk-Mapping

Betreuer	Christopher Scherb Luc Heitz
Student	Martin Vogt
Studiengang	Bachelor of Science FHNW in Informatik
Datum	22.02.2026

1 Einleitung

Durch die Digitalisierung und Vernetzung von IT-Infrastrukturen entstehen hochkomplexe Netzwerkarchitekturen mit dynamischen Kommunikationsbeziehungen. Insbesondere in heterogenen Umgebungen mit virtuellen Maschinen, Containern und Cloud-Komponenten erhöht sich die Zahl Angriffsflächen stark.

Ein wichtiger Ansatz zur Reduktion lateraler Bewegungen innerhalb eines Netzwerks ist die Microsegmentierung. Durch die feingranulare Definition von Kommunikationsregeln zwischen einzelnen Hosts oder logischen Segmenten wird das Prinzip der minimalen Berechtigung (Least Privilege) konsequent umgesetzt. Die manuelle Erstellung und Pflege entsprechender Regelwerke ist jedoch aufwendig, fehleranfällig und skaliert nur begrenzt mit wachsender Netzwerkgrösse.

Vor diesem Hintergrund untersucht dieses Projekt die Entwicklung einer automatisierten Lösung zur regelbasierten Microsegmentierung auf Basis eines Digital Twin Modells des Netzwerks. Durch die Abbildung von Netzwerkelementen als Graph mit formalisierter Darstellung von Kommunikationsbeziehungen soll eine algorithmische Synthese minimaler, sicherheitsoptimierter Regel-Sets ermöglicht werden. Ziel ist es, notwendige Kommunikation systematisch von nicht notwendiger oder potenziell riskanter Kommunikation zu trennen und eine Deny-by-Default-Strategie technisch konsistent umzusetzen.

2 Zielsetzung

Das Hauptziel dieses Projektes ist die Entwicklung einer automatisierten Lösung zur Microsegmentierung von Netzwerken basierend auf Digital Twin Technologie. Im Mittelpunkt steht die Konzeption und prototypische Implementierung eines Systems zur Generierung von Microsegmentierungsregeln auf Basis eines graphbasierten Netzwerkmodells.

2.1 Modellierung eines Netzwerks als Graph

Die Abbildung eines Netzwerks als Graph soll über die Darstellung von Hosts, Netzwerkgeräten und Segmenten als einzelne Knoten erreicht werden. Beziehungen zwischen den einzelnen Knoten sind als Kanten zu definieren, welche allenfalls weitere Attribute wie Protokolle, Ports oder VLANs beinhalten.

2.2 Sicherheitsorientierte Optimierung

Die automatische Analyse von Netzwerkverkehr soll neben der blossen Erkennung von Verbindungen und deren Zuteilung in geeignete Segmentierungen, auch die Möglichkeit bieten, zwischen notwendigem und überflüssigen Verbindungen zu unterscheiden. Darüber hinaus soll auch eine Bewertung und Deaktivierung von unsicheren Protokollen möglich sein.

2.3 Entwicklung eines Algorithmus zur Regel-Synthese

Es soll ein Algorithmus entwickelt werden, um ein minimales Regel-Set zu generieren, welches nur notwendige Kommunikation erlaubt, einen Deny-by-Default Ansatz verfolgt und eine maximale Isolation der verschiedenen Knoten erlaubt.

2.4 Prototypische Validierung

Der erstellte Prototyp kann neben reinen Code-Test-Cases in einer Testumgebung (z.B. Containerlab) geprüft und auf Auswirkungen bei unterschiedlichem Segmentierungsgrad analysiert werden. Die generierten Regel-Sets sollen in der Testumgebung appliziert und auf Funktionalität und Realisierbarkeit geprüft werden.

3 Methodik und Vorgehensweise

Im Verlauf der ersten Projektwochen wurde durch Gespräche und Recherchen entschieden, dass ein graphbasierter Ansatz zur automatisierten Microsegmentierung die Basis bilden soll. Diese Entscheidung basiert nicht nur auf der weiten Verbreitung dieser Technologien, sondern auch auf der bereits vorhandenen Erfahrung an der FHNW, die eine effiziente Implementierung und Betreuung erleichtert.

Anfang März liegt der Fokus auf der Literaturrecherche und der Analyse bestehender Technologien wie Segmentierungsansätze und Microsegmentierungsstrategien. Weiter gilt es die Anforderungen für das Zielsystem genau einzugrenzen und das Feature-Set zu definieren. Zusätzlich sind die Werkzeuge für die Implementierung zu evaluieren. Im Kickoff bereits vorgemerkt wurden NetworkX, Containerlab, Wireshark und Z3.

Mitte März bis Ende Juni wird das Konzept des Digital Twin Modells ausgearbeitet. Die Definition der Netzwerkelemente als Knotenklassen und Kommunikationsbeziehungen als Kanten mit Attributen steht im Zentrum. Im Architekturentwurf wird also das Graphmodell und eine formale Beschreibung der Regel-Synthese erstellt. Dabei wird der erste Prototyp des Systems entwickelt. Es geht um eine konkrete Implementierung des Digital Twin Modells und der Analysealgorithmen sowie der Regelgenerierung. Anschliessend folgt eine Test- und Validierungsphase im Juli.

Parallel dazu wird die Bewertung der Segmentierungsqualität, Analyse der Skalierbarkeit sowie die Diskussion von Grenzen und offenen Fragestellungen in die schriftliche Arbeit einfließen. Der Projektabschluss ist für August geplant, mit einer Reservezeit für letzte Anpassungen und die Fertigstellung der Dokumentation.

4 Zeitplan

Phase	Zeitraum	Beschreibung	Meilensteine
Projektstart und Planung ca. 10 Stunden	Februar (1 Woche)	- Klärung der groben Anforderungen - Erstellen eines groben Arbeitsplans	Projektvereinbarung fertig und angenommen
Analyse und Recherche ca. 40 Stunden	März (2 Wochen)	- Literaturrecherche - Analyse bestehender Technologien - Definition der Anforderungen	Definiertes Feature Set und Liste der einzusetzenden Tools
Konzeption und Implementierung ca. 220 Stunden	März bis Juni (11 Wochen)	- Definition des Graphmodells - Architekturentwurf - Beschreibung der Regel-Synthese - Implementierung des Digital Twin Modells - Implementierung der Analysealgorithmen - Entwicklung der Regelgenerierung	Erstellung eines Architekturentwurfs und Formalen Beschreibung der Regel-Synthese Erste funktionstfähige Version des Prototyps bereitstellen
Test und Validierung ca. 50 Stunden	Juli (2 Wochen)	- Aufbau der Containerlab-Umgebung - Durchführung der Testscenarien - Analyse und Optimierung	Ergebnisse des direkten Vergleichs analysieren, Optimierungen umsetzen
Dokumentation und Vorbereitung Präsentation ca. 40 Stunden	Juli (2 Wochen)	- Verfassen der schriftlichen Arbeit - Erstellung von Grafiken und Diagrammen - Vorbereitung der Präsentation und Verteidigung	Fristgerechte Abgabe aller Produkte

5 Risiken

5.1 Technische Komplexität

Die Modellierung realer Netzwerke ist sehr komplex sein.

Daher ist es wichtig, in einer frühen Phase die Eingrenzung des Betrachtungsbereichs vorzunehmen und eine klare Definition über den Scope des Systems anzulegen.

5.2 Skalierbarkeit des Algorithmus

Graphbasierte Berechnungen und Constraint-Solving sind rechenintensiv sein und allenfalls mit den zur Verfügung gestellten Mitteln nicht auf jede Problemgrösse angewendet werden.

Die Testphase soll sicherstellen, dass das System auf repräsentative Netzwerke angewendet werden kann. Eine sinnvolle Grösse der zu analysierenden Netze muss in Absprache mit den Projekt-Betreuern definiert werden.

5.3 Zeitmanagement

Eine Kombination aus Forschung, Implementierung und Dokumentation ist zeitintensiv und birgt Möglichkeiten für unvorhergesehenen Verzögerungen.

Eine strikte Meilensteinplanung und ein frühzeitiger Beginn der schriftlichen Arbeit sollen genutzt werden, um Verzug im Projekt vorzubeugen und Mängel frühzeitig zu erkennen.

5.4 Militärdienst

Das Spitalbataillon 5 der Logbrigade 1 führt den diesjährigen Wiederholungskurs vom 1. Juni 2026 bis und mit dem 26. Juni 2026 durch. Aufgrund von personellen Engpässen ist keine Dienstverschiebung für den Studenten Martin Vogt vorgesehen.

Während des Dienstes ist mit einer stark reduzierten Arbeitsleistung zugunsten dieses Projekts zu rechnen. Arbeitspakete müssen auf die anderen Projektwochen umverteilt werden.

6 Zustimmungserklärung

Mit ihrer Unterschrift bestätigen die unterzeichnenden Parteien, dass sie die vorliegende Projektvereinbarung gelesen haben und mit den definierten Zielen, Inhalten, Zeitplanung und Rahmenbedingungen einverstanden sind.

Das Projekt wird gemäss dieser Vereinbarung gestartet und durchgeführt.

Änderungen bedürfen der schriftlichen Zustimmung beider Parteien.

Martin Andreas Vogt
Student
22-479-299
martin.vogt1@students.fhnw.ch

Christopher Scherb
Haupt Betreuer
christopher.scherb@fhnw.ch

Datum / Unterschrift:



Datum / Unterschrift:



Meetingprotokoll

Datum: 25.03.2025

Teilnehmer

- Luc
- Christopher
- Martin

Besprechung

Aktueller Stand wurde mit Luc und Christopher besprochen.

Fortschritt

Von Martin wurden die folgenden Issues mit einem 30-Node-Netzwerk umgesetzt und erfolgreich getestet:

- Create prototype project structure
- Define network communication dataset format
- Create synthetic network dataset
- Implement csv traffic loader
- Implement digital twin graph builder
- Implement basic graph visualisation
- Implement graph metrics analysis

Neuer Ansatz

Es soll ein neuer Ansatz verfolgt werden:

- Start mit einem kleineren Netzwerk (~2–3 Switches und 8–10 User)
- Definition von „High-Level Constraints“ in einem geeigneten Datenformat

Beispiel Constraints

- User 1 und User 7 sind am gleichen Switch 01
- User 8 ist am Switch 02
- Switch 01 und Switch 02 sind verbunden

Kommunikationsregeln

- User 1 ↔ User 8: Kommunikation erlaubt
- User 8 ↔ User 7: Kommunikation erlaubt
- User 1 ↔ User 7: Kommunikation **nicht erlaubt**

Die Netzwerktopologie soll dabei berücksichtigt werden.

Ziel

Das System soll selbst erkennen:

- Wo VLANs eingesetzt werden müssen
- Wo ggf. zusätzliche Netzwerkkomponenten benötigt werden

Beispiel

- User 1 und User 8 müssen miteinander kommunizieren können
- → VLAN erforderlich
- → Alle Netzwerkgeräte zwischen den beiden Usern müssen dieses VLAN unterstützen

Einschränkungen

- Redundanzen müssen bei der Problemlösung nicht berücksichtigt werden

Offene Punkte

- Definition des minimalen Netzwerks zur Demonstration des Problems
- Erwartet wird keine vollständige Enterprise-Lösung, sondern ein Konzept

Meetingprotokoll

Datum: 07.04.2026

Teilnehmer

- Luc
- Christopher
- Martin

Besprechung

Der aktuelle Fortschritt wurde gemeinsam besprochen und als insgesamt zufriedenstellend bewertet.

Fortschritt

- Erste Variante der Heuristic Synthesis **ohne Z3 Solver** wurde implementiert und getestet
- Ergebnis: Funktionierender erster Ansatz als Grundlage für weitere Iterationen

Neuer Ansatz

Als nächster Schritt soll eine Variante **mit Solver-Unterstützung (Z3)** umgesetzt werden.

- Feature Set für den neuen Ansatz wurde erarbeitet
- Dokumentation:
https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/blob/main/docs/knowledge-base/ideas/feature_set_definition_v2.md
(https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/blob/main/docs/knowledge-base/ideas/feature_set_definition_v2.md)

Weiterführende Ressourcen

Folgende Inhalte wurden als potenziell hilfreich identifiziert:

- Advanced Topics in Communication Networks (BGP Fokus)
<https://adv-net.ethz.ch/transcripts/verification/> (<https://adv-net.ethz.ch/transcripts/verification/>)

Ziel: Relevante Konzepte und Methoden extrahieren und ggf. in den eigenen Ansatz integrieren.

Tools Evaluation

Luc hat eine Übersicht möglicher Tools zusammengestellt, die bis zum nächsten Meeting evaluiert werden sollen:

Goal	Best Tool
Flexible research modeling	Z3
Built-in synthesis	CVC5
Natural rule-based modeling	Clingo (ASP)
Scalable reachability analysis	Soufflé (Datalog)
Optimization-heavy problems	OR-Tools
Real network configs	Batfish

Nächste Schritte

- Implementierung eines Solver-basierten Ansatzes mit Z3
- Analyse und ggf. Integration von Erkenntnissen aus den BGP-Verifikationsmaterialien
- Evaluation der identifizierten Tools hinsichtlich Eignung für das Projekt

Offene Punkte

- Entscheidung für finalen Toolstack
- Abgrenzung des Feature Sets für die nächste Iteration

Meetingprotokoll

Datum: 21.04.2026

Teilnehmer

- Luc
- Christopher
- Martin

Besprechung

Der aktuelle Stand der Implementierung wurde präsentiert und gemeinsam diskutiert.

Der Fokus lag auf der Vorstellung der bisherigen Ergebnisse im Bereich Solver-Modellierung sowie der Einordnung des aktuellen Gesamtansatzes.

Fortschritt

- Durchführung von Test Runs zur Validierung des aktuellen Ansatzes
<https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/issues/25>
(<https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/issues/25>)
- Erstellung und Dokumentation eines Solver-Modells
https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/blob/main/docs/knowledge-base/ideas/solver_model.md (https://github.com/FHNW-Security-Lab-Student-Projects/ip6-automatisierte-microsegmentierung/blob/main/docs/knowledge-base/ideas/solver_model.md)
- Aktueller Stand der Segmentierungsansätze:
 - Constraint-basierte Segmentierung
 - Solver-basierter Ansatz (formal korrekt, optimierbar)
 - Heuristischer Ansatz (greedy)
- Struktur und Aufbau eines Digital Twins wurde definiert

Aktueller Ansatz

Der derzeitige Systementwurf basiert auf zwei komplementären Paradigmen:

- Constraint-basierte Microsegmentierung mittels SMT (Solver)
- Kombination aus:

- Graph Analytics
- Constraint Solving

Ziel ist es, sowohl formale Korrektheit als auch praktische Skalierbarkeit und Interpretierbarkeit zu erreichen.

Ideen & Weiteres Vorgehen

Feature-Erweiterungen

- Einführung von Node-Gruppen
- Definition von Regeln auf Gruppenebene
- Erweiterung des Constraint-Modells:
 - Neben „Allow“ und „Deny“ auch Service-basierte Regeln
 - Ermöglichen spezifischer Kommunikation über VLAN-Grenzen hinweg

Vergleich & Evaluation

Geplante Evaluationsdimensionen:

- Qualität der Segmentierung (z. B. Isolation)
 - Beispielvergleich:
 - Heuristik: 8 VLANs
 - Solver: 5 VLANs
- Laufzeit und Skalierbarkeit:
 - Tests mit:
 - 10 Nodes
 - 100 Nodes
 - 1000 Nodes
- Vollständige Erfüllung aller Constraints
- Anzahl benötigter VLANs

Analyse & Erweiterte Methoden

- Analyse von UNSAT-Fällen zur Identifikation von Konflikten im Constraint-System
- Einsatz von Community Detection zur Generierung von Segment-Vorschlägen
- Integration von Graph-Metriken in die Segmentierungslogik:
 - Degree Centrality:
 - Identifikation zentraler Systeme
 - Vermeidung unnötiger Isolation kritischer Nodes
 - Betweenness Centrality:
 - Identifikation kritischer Kommunikationsknoten

- gezielte Schutzmaßnahmen

Nächste Schritte

- Implementierung zusätzlicher Features (Gruppen, Service-basierte Regeln)
- Durchführung systematischer Evaluationen (Heuristik vs. Solver)
- Skalierungstests mit größeren Netzwerken
- Analyse und Behandlung von UNSAT-Fällen
- Integration von Graph-Analytics-Methoden in den Segmentierungsansatz

Offene Punkte

- Konkrete Definition und Modellierung von Service-basierten Constraints
- Methodik zur Kombination von Graph Analytics und Constraint Solving
- Bewertungskriterien für „optimale“ Segmentierung
- Umgang mit Zielkonflikten (z. B. Sicherheit vs. Funktionalität)

Meetingprotokoll

Datum: 12.05.2026

Teilnehmer

- Luc
- Christopher
- Martin

Besprechung

Der aktuelle Fortschritt der Implementierung sowie erste Evaluationen wurden vorgestellt und diskutiert. Der Schwerpunkt lag auf der Analyse von UNSAT-Fällen, Performance-Messungen sowie der Weiterentwicklung der Netzwerkmodellierung und Gruppenlogik.

Zusätzlich wurden mögliche Optimierungen des Constraint-Modells sowie Visualisierungs- und Evaluationsideen für kommende Iterationen besprochen.

Fortschritt

- Analyse und Behandlung von UNSAT-Fällen implementiert
- Durchführung von Tests mit Netzwerken unterschiedlicher Größe:
 - 60 Hosts
 - 80 Hosts
 - 100 Hosts
- Messung der Laufzeiten während der Solver-Ausführung
- Implementierung eines Network Builders
- Einführung und Umsetzung eines Gruppen-Features

Aktueller Ansatz

Der aktuelle Segmentierungsansatz kombiniert weiterhin:

- Constraint-basierte Segmentierung mittels Solver
- Heuristische Segmentierung
- Netzwerkmodellierung über Graphstrukturen

Die Implementierung unterstützt inzwischen Gruppen-basierte Modellierungen und größere Netzwerkszenarien zur Evaluation von Skalierbarkeit und Laufzeitverhalten.

Ideen & Weiteres Vorgehen

Visualisierung von UNSAT-Fällen

Bei nicht lösbaren Netzwerkkonfigurationen könnte dennoch ein physischer Netzwerk-Graph erzeugt werden, um Konflikte visuell darzustellen.

Mögliche Erweiterungen:

- Markierung problematischer Nodes und Kommunikationspfade
- Hervorhebung von Konflikten mittels Farbcodierung (z. B. rot)
- Integration der Solver-/Terminal-Fehlermeldungen in die Visualisierung

Ziel ist eine bessere Nachvollziehbarkeit von Constraint-Konflikten und deren Ursachen.

Erweiterte Performance-Analyse

Weitere Evaluationen wurden diskutiert:

- Untersuchung des Laufzeitverhaltens bei steigender Anzahl von Constraints
- Entwicklung eines Helpers zur automatischen Generierung von Netzwerken mit definierter Constraint-Anzahl
- Vergleich des Skalierungsverhaltens verschiedener Segmentierungsansätze

Optimierung der Gruppenmodellierung

Ein möglicher Optimierungsansatz besteht darin, Gruppen im Constraint-System als einzelne abstrahierte Knoten zu behandeln.

Dadurch könnte vermieden werden, dass zwischen allen Gruppenmitgliedern vollständige Constraint-Graphen erzeugt werden.

Erwartete Vorteile:

- Reduktion der Constraint-Anzahl
- Verbesserte Skalierbarkeit
- Geringere Solver-Laufzeiten

Erweiterung der Optimierungsziele

Diskutiert wurde eine zusätzliche Betriebsoption:

- „Erfülle alle Constraints und segmentiere maximal“

Ziel wäre eine möglichst restriktive Segmentierung unter vollständiger Einhaltung aller definierten Regeln.

Edge-Case-Analysen

Zur Untersuchung möglicher Schwächen der Heuristik sollen spezielle Netzwerkstrukturen erzeugt werden:

- Kettenstrukturen
- Ringstrukturen

Dadurch könnten Szenarien identifiziert werden, die:

- vom Solver korrekt gelöst werden,
- vom heuristischen Ansatz jedoch nicht optimal behandelt werden.

Nächste Schritte

- Erweiterung der Visualisierung für UNSAT-Fälle
- Entwicklung automatisierter Constraint-Generatoren für Performance-Tests
- Optimierung der Gruppenmodellierung im Constraint-System
- Durchführung weiterer Skalierungs- und Laufzeittests
- Analyse von Edge Cases mit speziellen Netzwerkstrukturen
- Vorbereitung des Midterm Meetings am 16.06.2026

Midterm Meeting

Das Midterm Meeting findet am 16.06.2026 statt.

Der genaue Zeitpunkt muss noch definiert werden, damit die Planung während des Militärdienstes berücksichtigt werden kann.

Geplante Inhalte des Meetings:

- Strukturierte Präsentation des aktuellen Projektstands
- Vorstellung der bisherigen Ergebnisse
- Diskussion erfolgreicher und problematischer Aspekte des Projekts
- Überblick über geplante nächste Entwicklungsschritte

Offene Punkte

- Konkrete Umsetzung der Visualisierung von UNSAT-Konflikten
- Effiziente Modellierung von Gruppen im Constraint-System
- Definition geeigneter Metriken für maximale Segmentierung
- Vergleich der Skalierbarkeit zwischen Solver- und Heuristik-Ansatz
- Identifikation weiterer relevanter Edge Cases für die Evaluation

Meetingprotokoll

Datum: 07.07.2026

Teilnehmer

- Luc
- Christopher
- Martin

Besprechung

Im Meeting wurde der aktuelle Stand des Dokuments der Bachelorarbeit besprochen. Der Fokus lag auf der Struktur der Arbeit, der Überarbeitung einzelner Kapitel sowie den formalen Anforderungen an die finale Abgabe.

Besprochene Punkte

Dokumentstruktur

- Ein Glossar ist nicht zwingend erforderlich.
- Nach jeder Kapitelüberschrift soll ein kurzer einleitender Text folgen, der den Inhalt des Kapitels beschreibt. Reine Überschriften ohne Einleitung sollen vermieden werden.
- Der Abschnitt „**Aufbau der Arbeit**“ soll an das Ende des Kapitels **Einführung** verschoben werden.
- Das Kapitel **Related Work** soll überarbeitet werden. Mögliche Optionen:
 - eigenes Kapitel,
 - Integration in die Grundlagen,
 - oder Eingliederung in den Stand der Technik.
- Das Kapitel **Grundlagen** soll sich an Studierende mit vergleichbarem Wissensstand richten und bei Bedarf im weiteren Verlauf der Arbeit ergänzt werden.
- Das Kapitel **Systemdesign** kann gegebenenfalls in **Konzept** umbenannt werden.

Formale Anforderungen

- Für Zitate und Referenzen soll der IEEE-Standard verwendet werden.

Termine

- **Abgabe der Bachelorarbeit:** 07.08.2026, 23:59 Uhr
 - Abgabe als ZIP-Datei per E-Mail.
- **Präsentation und Poster:**

- Weitere Informationen folgen zum entsprechenden Zeitpunkt.

Nächste Schritte

- Dokumentstruktur entsprechend den besprochenen Punkten überarbeiten.
- Kapitel Einführung anpassen und Abschnitt „Aufbau der Arbeit“ verschieben.
- Entscheidung über die Einordnung von „Related Work“ treffen.
- Grundlagenkapitel bei Bedarf erweitern.
- Zitierstil auf IEEE umstellen.
- Bachelorarbeit bis spätestens 07.08.2026 fertigstellen und als ZIP per E-Mail einreichen.

Offene Punkte

- Finale Entscheidung zur Platzierung von „Related Work“.
- Entscheidung, ob das Kapitel „Systemdesign“ in „Konzept“ umbenannt wird.
- Informationen zu Präsentation und Poster abwarten.

Meetingprotokoll

Datum: 04.08.2026

Teilnehmer

- Christopher
- Martin

Besprechung

Im Meeting wurde der aktuelle Stand der Bachelorarbeit besprochen. Der Fokus lag auf der fachlichen Einordnung der implementierten Ansätze, der Aussagekraft der Evaluation sowie auf noch fehlenden Inhalten und formalen Bestandteilen der Arbeit.

Besprochene Punkte

Gruppenmodellierung

- Die aktuelle Gruppenmodellierung ist aufgrund der quadratischen Expansion der Gruppenbeziehungen nicht ausreichend effizient.
- Gruppen wurden deshalb in den Experimenten nicht verwendet.
- In der Arbeit soll klarer beschrieben werden, dass die aktuelle Implementierung bei grösseren Gruppen zu einer starken Zunahme der erzeugten Constraints führt und deshalb nur eingeschränkt skalierbar ist.

Besonderheiten und Designentscheidungen

- Die wesentlichen Besonderheiten und Designentscheidungen der entwickelten Lösung sollen deutlicher beschrieben werden.
- Dabei sollen insbesondere getroffene Vereinfachungen, technische Abwägungen und deren Auswirkungen auf die Resultate erläutert werden.

Heuristischer Ansatz

- Die Aussagekraft der Heuristik ist eingeschränkt, da sie zahlreiche Szenarien nicht erfolgreich lösen kann.
- Dieser Umstand soll in der Evaluation klar dargestellt und kritisch analysiert werden.
- Die Ergebnisse der Heuristik dürfen nicht isoliert anhand der Laufzeit beurteilt werden, sondern müssen gemeinsam mit der Erfolgsrate und der Gültigkeit der erzeugten Segmentierungen betrachtet werden.

Z3-Solver

- Die Optimierung des Z3-Solvers zur Minimierung der verwendeten VLANs soll beschrieben und ausgewertet werden.
- Es soll erläutert werden, wie sich die Optimierung auf die Anzahl der VLANs und die Laufzeit des Solvers auswirkt.

UNSAT-Fälle

- Nicht lösbare Szenarien mit dem Ergebnis **UNSAT** sollen genauer untersucht werden.
- In der Arbeit soll analysiert werden, welche Kombinationen aus Topologie und Kommunikationsanforderungen zu widersprüchlichen Constraints führen.
- Nach Möglichkeit sollen ausgewählte UNSAT-Fälle beispielhaft erklärt werden.

Anhang

Für den Anhang beziehungsweise die ergänzenden Verzeichnisse sollen folgende Inhalte geprüft und aufgenommen werden:

- Hilfsmittelverzeichnis
 - Verwendung von ChatGPT
 - Verwendung des Overleaf AI Assistant
- Meetingprotokolle
- Planung und Projektvereinbarung
- Glossar
- Abkürzungsverzeichnis

Organisatorisches

- Michael Faes ist als zuständige Ansprechperson für alle Module festgehalten.
- Alle offenen Verbesserungsvorschläge aus dem Kapitel „**Future Work**“ wurden bereits als GitHub Issues erfasst.

Termine

- **Abgabe der Bachelorarbeit:** 07.08.2026, 23:59 Uhr
- **Präsentation der Bachelorarbeit:** 01.09.2026, 09:00 Uhr

Nächste Schritte

- Abschnitt zur Gruppenmodellierung überarbeiten und die eingeschränkte Skalierbarkeit der aktuellen Implementierung erläutern.
- Besonderheiten und zentrale Designentscheidungen der Lösung dokumentieren.
- Aussagekraft und Erfolgsrate der Heuristik kritisch analysieren.
- Optimierung des Z3-Solvers zur VLAN-Minimierung beschreiben und auswerten.
- Ausgewählte UNSAT-Fälle untersuchen und in der Arbeit erläutern.

- Hilfsmittelverzeichnis und weitere Bestandteile des Anhangs ergänzen.
- Bachelorarbeit bis spätestens 07.08.2026, 23:59 Uhr fertigstellen und abgeben.
- Präsentation für den 01.09.2026 um 09:00 Uhr vorbereiten.

Offene Punkte

- Auswahl geeigneter UNSAT-Fälle für die detaillierte Analyse.
- Umfang der Beschreibung zur VLAN-Minimierung durch den Z3-Solver.
- Finale Zusammenstellung und Reihenfolge der Bestandteile des Anhangs.
- Klärung, ob Glossar und Abkürzungsverzeichnis beide benötigt werden.